PROJECT REPORT

ON

Network Intrusion Detection Using Ensemble Classifier

A Dissertation Submitted

In partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

S. Durga Sai Archit **17211A12A4**

Under the Esteemed Guidance of

V. Rupesh,

M. Tech (Ph.D.), Assistant Professor

Department of Information Technology

DEPARTMENT OF INFORMATION TECHNOLOGY

B. V. RAJU INSTITUTE OF TECHNOLOGY (UGC Autonomous)

Vishnupur, Narsapur, Medak (Dist.) - 502313 (TS)

(Affiliated to JNTU and approved by A.I.C.T.E)

2017-2021

B.V.Raju Institute of Technology
(UGC Autonomous)
Vishnupur, Narsapur, Medak (Dist)-502313(TS)
www.bvrit.ac.in

DECLARATION

I here by declare that the project entitled **"Network Intrusion Detection Using Ensemble Classifier" submitted** to the Department of Information Technology, B.V.Raju Institute of Technology, in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology** in **Information Technology** is a record of the original work done by me and has not been submitted to any institute or published elsewhere.

S Durga Sai Archit **17211A12A4**

Place: Narsapur

Date:

B.V.Raju Institute of Technology
(UGC Autonomous)
Vishnupur, Narsapur, Medak (Dist)-502313(TS)
www.bvrit.ac.in

CERTIFICATE FROM INTERNAL GUIDE & CONTROLLER

This is to certify that **S Durga Sai Archit** of B.Tech final year from Information Technology Department, B.V. Raju Institute Of Technology have successfully completed the project entitled **"Network Intrusion Detection Using Ensemble Classifier"** in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology** in **Information Technology** under my supervision.

His performance during this period was commendable and I wish him all the best for the future.

Guide
Mr. V. Rupesh
M.Tech (Ph.D.)
Assistant Professor
Department of IT

Project Controller
Dr M Neelakantapa
M.Tech, Ph.D
Professor
Department of IT

B.V.Raju Institute of Technology
(UGC Autonomous)
Vishnupur, Narsapur, Medak (Dist)-502313(TS)
www.bvrit.ac.in

CERTIFICATE FROM PROJECT COORDINATOR

This is to certify that **S Durga Sai Archit** of B.Tech final year from Information Technology Department, B.V.Raju Institute Of Technology have successfully completed the project entitled **"Network Intrusion Detection Using Ensemble Classifier"** in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology** in **Information Technology** and this thesis is a bonafide record of the work done in this project.

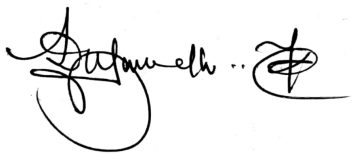
His performance during this period was commendable and I wish him all the best for the future.

Mrs. Divanu Sameera
M.Tech (Ph.D)
Associate Professor
Department of IT

B.V.Raju Institute of Technology
(UGC Autonomous)
Vishnupur, Narsapur, Medak (Dist)-502313(TS)
www.bvrit.ac.in

CERTIFICATE FROM HEAD OF THE DEPARTMENT

This is to Certify that the project entitled **"Network Intrusion Detection Using Ensemble Classifier"** That is being submitted by **S Durga Sai Archit** bearing roll number **17211A12A4** in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology** in **Information Technology** is a bonafide record of the work carried out by him.



Dr. K. Dasaradha Ramaiah
M.Tech, Ph.D
Professor & HOD
Department of IT

B.V.Raju Institute of Technology
(UGC Autonomous)
Vishnupur, Narsapur, Medak (Dist)-502313(TS)
www.bvrit.ac.in

CERTIFICATE

This is to Certify that the project entitled **"Network Intrusion Detection Using Ensemble Classifier"** being submitted in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology** in **Information Technology** to the Jawaharlal Nehru Technological University, Hyderabad is a bonafide work carried out by team under the guidance of **Mr. V. Rupesh, Assistant Professor**, Department of Information Technology.

The project report is submitted by the team

S Durga Sai Archit **17211A12A4**

(Internal Examiner) (External Examiner)

ACKNOWLEDGEMENT

This acknowledgement transcends the reality of formality when I would express deep gratitude and respect to all those people who helped supported, guided and inspired me throughout the completion of this project.

I would like to express my deep gratitude and wholehearted thanks to my guide **Mr. V. Rupesh, M.Tech (Ph.D.), (Assistant Professor, Department of Technology, B.V. Raju Institute of Technology)** for his valuable guidance, precious suggestions, and encouragement in completing the project successfully.

I would like to express my deep gratitude and wholehearted thanks to **Dr M Neelakantapa, M.Tech, Ph.D.** (**Project Controller, Department of Information Technology, B.V. Raju Institute of Technology**) for his valuable guidance, precious suggestions, and encouragement in completing the project successfully.

I express my sincere thanks to **Mrs. Divanu Sameera, M. Tech (Ph. D) (Project Coordinator, Department of Information Technology, B.V. Raju Institute of Technology)** who played important part, worked to ensure **projects** are completed on time and are primarily responsible for administrative tasks providing valuable guidance.

I wish to convey my sincere thanks to **Dr. R V R Chary, M.Tech, Ph. D (I/C HOD, Dept of IT, B.V. Raju Institute of Technology)** for his continuous support.

I wish to convey my sincere thanks to **Dr. K Dasaradha Ramaiah, M.Tech, Ph. D (HOD, Dept of IT, B.V. Raju Institute of Technology)** for his continuous support.

I wish to convey my sincere thanks **to Dr. K. Lakshmi Prasad, M.Tech (IIT Bombay), Ph. D (Principal, B.V. Raju Institute of Technology)** for his continuous support.

I am also grateful to all administrative staff and the technical staff of Information Technology Department, people who provided me the useful and necessary information needed for this project. Lastly, I thank all my family, friends, and well-wishers.

S Durga Sai Archit (17211A12A4)

ABSTRACT

Network Intrusion Detection Systems (NIDSs) are essential tools for the network system administrators to detect various security breaches inside an organization's network. A NIDS monitors and analyses the network traffic entering or exiting from the network devices of an organization and raises alarms if an intrusion is observed.

Intrusion detection system (IDS) is one of the extensively used techniques in a network topology to safeguard the integrity and availability of sensitive assets in the protected systems. However, many challenges arise while developing a flexible and efficient NIDS for unforeseen and unpredictable attacks. We propose a machine learning-based approach for developing such an efficient and flexible NIDS.

We propose to apply ensemble-based algorithms like random forest or xgboost algorithms. Ensemble methods are meta-algorithms that combine several machine learning techniques into one predictive model to decrease variance (bagging), bias (boosting), or improve predictions (stacking). Unlike a statistical ensemble in statistical mechanics, which is usually infinite, a machine learning ensemble consists of only a concrete finite set of alternative models, but typically allows for much more flexible structure to exist among those alternatives. This method promises to provide greater accuracy than existing method named SVM. This method is also faster and more stable in the real time.

LIST OF CONTENTS

Chapter 1: INTRODUCTION 1

1.1 Intrusion Detection Dataset 3

1.1.1 DAPRA/ KDD CUP99 dataset 3

1.1.2 NSL-KDD Dataset 4

1.1.3 ADFA-LD Dataset 5

1.1.4 CICIDS 2017..... 6

1.2 Comparison of public IDS datasets 6

1.3 Technologies 9

1.3.1 PYTHON 9

1.3.2 NUMPY 10

1.3.3 SCIKIT LEARN 10

1.3.4 PANDAS 10

1.3.5 PICKLE 11

1.4 System Requirements 12

Chapter 2: LITERATURE SURVEY 13

Chapter 3: EXISTING SYSTEM & PROPOSED SYSTEM 18

3.1 Existing system 18

3.2 Proposed system 19

Chapter 4: ALGORITHMS 21

4.1 Ensemble Classifier 21

4.1.1 Sequential approach 21

4.1.2 Parallel approach 21

4.1.3 Bagging 22

4.1.4 Boosting ……..…… 23

4.1.5 Voting Classifier ……..... 23

4.2 Xgboost ……..... 24

4.3 Random Forest Classifier ……..... 26

4.4 Extra Trees Classifier ……..... 27

Chapter 5: SYSTEM DESIGN & UML DESIGN …….. 29

5.1 Unified Modeling Language (UML) ……..….... 30

5.1.1 An Overview of UML ……..... 30

5.1.2 A Conceptual Model of UML ……..... 30

5.1.3 Basic Building Blocks of The UML ……..…… 31

5.1.4 Things in The UML ……..….. 31

5.1.5 Relationships in The UML ……..…... 31

5.2 UML Diagrams ……..... 32

5.2.1 Use Case diagram ……..... 32

5.2.2 Communication diagram ……... 33

5.2.3 Activity diagram ……..... 36

5.2.4 Sequence diagram ……..... 40

5.2.5 Class diagram ……..... 41

CHAPTER 6: SOURCE CODE ...……...……..... 43

6.1 Data Module ……...……...…… 43

6.2 Detector Module ……..... 45

6.3 Pre-processor Module ……..... 47

6.4 Model Training ……..... 50

6.5 Train Data Module ……..... 53

6.6 Model Controller ……... 55

6.7 GUI ……..…… 55

CHAPTER 7: IMPLEMENTATION ……..... 60

7.1 Dataset Loading and Pre-processing ……... 60

7.2 Model Training …… 61

7.3 Comparison of Different Models Based on Accuracies …….. 62

7.4 GUI Output For Intrusion Detected ……..... 62

7.5 GUI Output For Report …… 63

CHAPTER 8: TESTING ……..... 64

8.1 Introduction …….. 64

8.2 Model Testing On a Set of Processed Values …….. 65

8.3 GUI Testing …….. 65

CHAPTER 9: CONCLUSION ……..... 66

CHAPTER 10: BIBLIOGRAPHY ……..... 67

CHAPTER 11: APPENDIX ……. 69

LIST OF FIGURES

S.no	***Figure number***	***Figure name***	***Page number***
1	Fig 1.1	Types of IDS Classification	2
2	Fig 1.1.1	KDD CUP99	4
3	Fig 1.1.2	Features of ADFA – LD dataset	5
4	Fig 1.2.1	Characteristics of the datasets	6
5	Fig 1.2.2	Summary of the datasets	9
6	Fig 3.1.1	A training example of SVM	19
7	Fig 3.2.1	Ensemble Learning	20
8	Fig 4.1	Bagging Classifier	22
9	Fig 4.2	Boosting Classifier	23
10	Fig 4.3	Voting Classifier	24
11	Fig 4.4	Xgboost Classifier	25
12	Fig 4.5	Random Forest Classifier	26
13	Fig 4.6	Extra Trees Classifier	28
14	Fig 5.1	Use Case Diagram	33
15	Fig 5.2	Communication diagram for training	35
16	Fig 5.3	Communication diagram for detection	36
17	Fig 5.4	Activity diagram for model training	38
18	Fig 5.5	Activity diagram for Detection	39
19	Fig 5.6	Sequence Diagram for Network Intrusion Detection	40
20	Fig 5.7	Class diagram for training mode	42
21	Fig 5.8	Class diagram for Detection	42
22	Fig 7.1	Dataset Loading and Pre-processing	60
23	Fig 7.2	Model Training	61
24	Fig 7.3	Comparison of Models	62
25	Fig 7.4	GUI Output For Intrusion Detected	62
26	Fig 7.5	GUI Output For Report	63
27	Fig 8.1	Model Testing	65
28	Fig 8.2	GUI Testing	65

Chapter 1

INTRODUCTION

A network is a group of nodes interconnected using protocols for the purpose of sharing resources. The interconnections are formed from a broad spectrum of telecommunication network technologies, based on physically wired, optical, and wireless radio-frequency methods that may be arranged in a variety of network topologies. The nodes may include personal computers, servers, networking hardware, or other specialised or general-purpose hosts. They are identified by hostnames and network addresses. Hostnames serve as memorable labels for the nodes, rarely changed after initial assignment. Network addresses serve for locating and identifying the nodes by communication protocols such as the Internet Protocol.

The network connections are usually prone to many kinds of virus and malpractices. These are generally regarded as intrusions or attacks on the networks. These intrusions are very dangerous as the intruders try to get our private information through this process and use it against us. However, intrusion detection systems can help in solving this problem.

Intrusion detection system (IDS) is one of the extensively used techniques in a network topology to safeguard the integrity and availability of sensitive assets in the protected systems. A Network Intrusion Detection System (NIDS) helps system administrators to detect network security breaches in their organizations. However, many challenges arise while developing a flexible and efficient NIDS for unforeseen and unpredictable attacks.

IDS can be classified into following categories,

- detection takes place,
- detection method that is employed

The location-based IDS can be either network based, or host based. The detection-based can be either signature/ misuse based, or anomaly based.

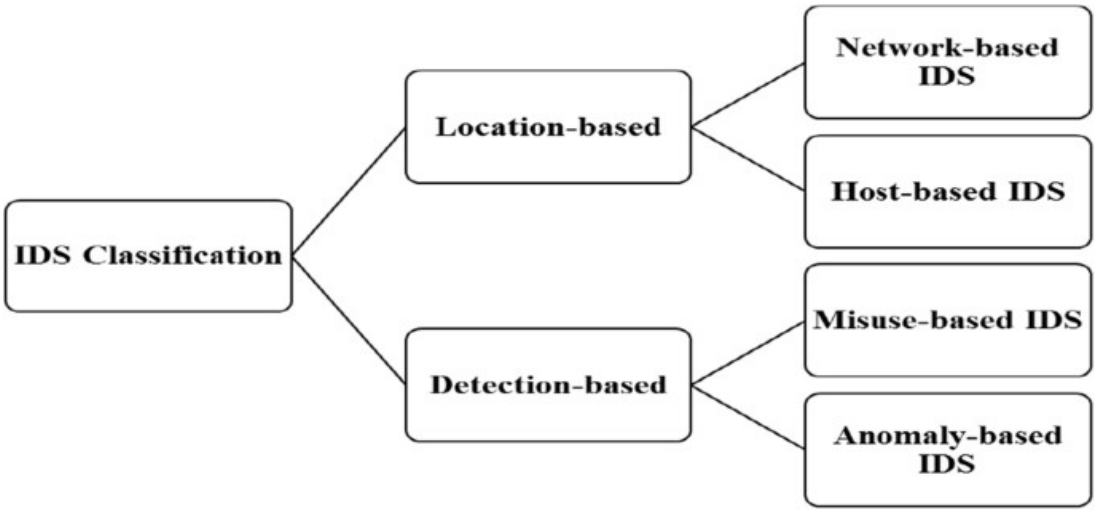


Fig 1.1 Types of IDS Classification

Network Based IDS

Network intrusion detection systems (NIDS) are placed at a strategic point or points within the network to monitor traffic to and from all devices on the network. It performs an analysis of passing traffic on the entire subnet and matches the traffic that is passed on the subnets to the library of known attacks. Once an attack is identified, or abnormal behaviour is sensed, the alert can be sent to the administrator. An example of an NIDS would be installing it on the subnet where firewalls are located in order to see if someone is trying to break into the firewall. Ideally one would scan all inbound and outbound traffic, however doing so might create a bottleneck that would impair the overall speed of the network. OPNET and NetSim are commonly used tools for simulating network intrusion detection systems.

Host Based IDS

Host intrusion detection systems (HIDS) run on individual hosts or devices on the network. A HIDS monitors the inbound and outbound packets from the device only and will alert the user or administrator if suspicious activity is detected. It takes a snapshot of existing system files and matches it to the previous snapshot. If the critical system files were modified or deleted, an alert is sent to the administrator to investigate. An example of HIDS usage can be seen on mission critical machines, which are not expected to change their configurations.

Signature Based IDS

Signature-based IDS refers to the detection of attacks by looking for specific patterns, such as byte sequences in network traffic, or known malicious instruction sequences used by malware. This terminology originates from anti-virus software, which refers to these detected patterns as signatures. Although signature-based IDS can easily detect known attacks, it is difficult to detect new attacks, for which no pattern is available. In Signature-based IDS, the signatures are released by a vendor for its all products. On-time updating of the IDS with the signature is a key aspect.

Anomaly Based IDS

Anomaly-based intrusion detection systems were primarily introduced to detect unknown attacks, in part due to the rapid development of malware. The basic approach is to use machine learning to create a model of trustworthy activity, and then compare new behaviour against this model. Since these models can be trained according to the applications and hardware configurations, machine learning based method has a better generalized property in comparison to traditional signature-based IDS. Although this approach enables the detection of previously unknown attacks, it may suffer from false positives: previously unknown legitimate activity may also be classified as malicious. Most of the existing IDSs suffer from the time-consuming during detection process that degrades the performance of IDSs. Efficient feature selection algorithm makes the classification process used in detection more reliable.

1.1 Intrusion Detection Dataset

The evaluation datasets play a vital role in the validation of any IDS approach, by allowing us to assess the proposed method's capability in detecting intrusive behaviour. There are a few publicly available datasets such as DARPA, KDD, NSL-KDD and ADFA-LD and they are widely used as benchmarks.

1.1.1 DARPA / KDD CUP99 dataset

The earliest effort to create an IDS dataset was made by DARPA (Defence Advanced Research Project Agency) in 1998 and they created the KDD98 (Knowledge Discovery and Data Mining (KDD)) dataset. Although this dataset was an important contribution to the research on IDS, its accuracy and capability to consider real-life conditions have been widely criticized.

The 1998 DARPA Dataset was used as the basis to derive the KDD Cup99 dataset which has been used in Third International Knowledge Discovery and Data Mining Tools Competition (KDD, 1999). These datasets are out-of-date as they do not contain records of recent malware attacks. Nevertheless, KDD99 remains in use as a benchmark within IDS research community and is still presently being used by researchers.

Label	Network data feature	Label	Network data feature	Label	Network data feature	Label	Network data feature
A	duration	L	Logged in	W	count	AH	dst_host_same_srv_rate
B	protocol-type	M	num_ comprised	X	srv_count	AI	dst_host_diff_srv_rate
C	service	N	root_shell	Y	serror_rate	AJ	dst_host_same_src_port_rate
D	flag	O	Stu attempted	Z	srv_serror_rate	AK	dst_host_srv_diff_host_rate
E	src_bytes	P	num_root	AA	rerror_rate	AL	dst_host_serror_rate
F	dst_bytes	Q	Num of file	AB	srv_rerror_rate	AM	dst_host_srv_serror_rate
G	land	R	Number of shell	AC	same_srv_rate	AN	dst_host_rerror_rate
H	wrong_fragment	S	num_access_files	AD	diff_srv_rate	AO	dst_host_srv_rerror_rate
I	urgent	T	num_outbound_cmds	AE	srv_diff_host_rate		
J	hot	U	Is host login	AF	dst_host_count		
K	num_falied_logins	V	Is guest login	AG	dst_host_srv_count		

Fig 1.1.1 KDD CUP99 Dataset

1.1.2 NSL – KDD Dataset

NSL-KDD is a public dataset, which has been developed from the earlier KDD cup99 dataset. The main problem in the KDD data set is the huge number of duplicate packets. This huge quantity of duplicate instances in the training set would influence machine-learning methods to be biased towards normal instances and thus prevent them from learning irregular instances which are typically more damaging to the computer system. The NSL-KDD train dataset consists of 125,973 records and the test dataset contains 22,544 records. The size of the NSL-KDD dataset is sufficient to make it

practical to use the whole NSL-KDD dataset without the necessity to sample randomly. This has produced consistent and comparable results from various research works.

1.1.3 ADFA – LD Dataset

Researchers at the Australian Defence Force Academy created two datasets (ADFA-LD and ADFA-WD) as public datasets that represent the structure and methodology of the modern attacks (Creech, 2014). The datasets contain records from both Linux and Windows operating systems. The ADFA – LD dataset comprises of three dissimilar data categories, each group of data containing raw system call traces. Each training dataset was gathered from the host for normal activities, with user behaviours ranging from web browsing to LATEX document preparation. Table shows some of the ADFA-LD features with the type and the description for each feature.

Name	Type	Description
srcip	nominal	Source IP address
sport	integer	Source port number
dstip	nominal	Destination IP address
dsport	integer	Destination port number
proto	nominal	Transaction protocol
state	nominal	Indicates to the state and its dependent protocol
dur	Float	Record total duration
sbytes	Integer	Source to destination transaction bytes
dbytes	Integer	Destination to source transaction bytes
sttl	Integer	Source to destination time to live value
dttl	Integer	Destination to source time to live value
sloss	Integer	Source packets retransmitted or dropped
dloss	Integer	Destination packets retransmitted or dropped
service	nominal	http, ftp, smtp, ssh, dns, ftp-data ,irc and (-) if not much used service
Sload	Float	Source bits per second
Dload	Float	Destination bits per second
Spkts	integer	Source to destination packet count
Dpkts	integer	Destination to source packet count
swin	integer	Source TCP window advertisement value
dwin	integer	Destination TCP window advertisement value
stcpb	integer	Source TCP base sequence number
dtcpb	integer	Destination TCP base sequence number
smeansz	integer	Mean of the how packet size transmitted by the src
dmeansz	integer	Mean of the how packet size transmitted by the dst
trans_depth	integer	Represents the pipelined depth into the connection of http request/response transaction
res_bdy_len	integer	Actual uncompressed content size of the data transferred from the server's http service.

Fig 1.1.2 Features of ADFA – LD dataset

1.1.4 CICIDS 2017

CICIDS2017 dataset comprises both benign behaviour and details of new malware attacks: such as Brute Force FTP, Brute Force SSH, DoS, Heartbleed, Web Attack, Infiltration, Botnet and DDoS. This dataset is labelled based on the timestamp, source and destination IPs, source and destination ports, protocols, and attacks. A complete network topology was configured to collect this dataset which contains Modem, Firewall, Switches, Routers, and nodes with different operating systems (Microsoft Windows (like Windows 10, Windows 8, Windows 7, and Windows XP), Apple's macOS iOS, and open-source operating system Linux). This dataset contains 80 network flow features from the captured network traffic.

1.2 Comparison of public IDS datasets

Since machine learning techniques are applied in AIDS, the datasets that are used for the machine learning techniques are very important to assess these techniques for realistic evaluation. Table 1.2.1 summarizes the characteristics of the datasets. Table 1.2.2 summarises popular public data sets, as well as some analysis techniques and results for each dataset from prior research.

Dataset	Realistic Traffic	Label data	IoT traces	Zero-day attacks	Full packet captured	Year
DARPA 98	√	√	X	X	√	1998
KDDCUP 99	√	√	X	X	√	1999
CAIDA	√	X	X	X	X	2007
NSL-KDD	√	√	X	X	√	2009
ISCX 2012	√	√	X	X	√	2012
ADFA-WD	√	√	X	√	√	2014
ADFA-LD	√	√	X	√	√	2014
CICIDS2017	√	√	X	√	√	2017
Bot-IoT	√	√	√	√	√	2018

Fig 1.2.1 Characteristics of the datasets

Dataset	Result	Observations	Reference
DARPA 98	Snort's detection, 69% of total generated alerts are considered to be false alarms.	SIDS is applied without AIDS	Hu, et al. (2009)
	ANN analysis system calls, 96% detection rate.	A classifier based on artificial neural network (ANN) has been executed for preparing and testing of framework.	McHugh (2000)
	SVM on subset of DARPA 98, 99.6% detection rate.	SVM isolates information into various classes by a hyperplane or hyperplanes since it can deal with multidimensional information. SVM usually demonstrate good performance for a binary class problem.	Chen, et al. (2005)
KDDCUP 99	Multivariate statistical analysis of audit data, 90% detection rate	Multivariate is used to reduce false alarm rates.	Ye, et al. (2002), Hotta, et al. (2008)
	The best results have been achieved by the C4.5 algorithm which attains the 95% true positive rate.	The decision trees created by C4.5 can be utilized for classification	Ferrari and Cribari-Neto (2004); Shafi and Abbass (2013); Laskov, et al. (2005)
	SMO classifier 97% detection rate.	This SVM based classifier with SMO implementation produces good detection accuracy. However, the accuracy reported is less than that in (Chen et al., 2005), because the KDDCUP 99 dataset is more complex and comprehensive than DARPA 98 dataset.	Shafi and Abbass (2013)

Dataset	Result	Observations	Reference
	The best model is an HNB model, where 95% confidence level is used to compare the models.	Hidden Naïve Bayes (HNB) techniques could be applied to IDS area that suffer from dimensionality, highly associated attributes, and high network speed. HNB technique is better than the one based on the traditional NB method in terms of detection accuracy for IDS.	Koc, et al. (2012)
NSL-KDD	K-Nearest Neighbour (k-NN) algorithm, the detection rate of 94%.	The k-NN algorithm uses all labelled training instances as a model of the target function. During the classification phase, k-NN uses a similarity-based search strategy to determine a locally optimal hypothesis function.	Adebowale, et al. (2013)
	Naïve Bayes, the detection rate is 89%.	Bayesian classifiers provide moderate accuracy because the focus is on classifying the classes for the instances, not the exact probabilities.	Adebowale, et al. (2013)
	C4.5 gave the best detection rate of 99%.	C4.5 selects the feature of the data that most efficiently divides its set of samples into subsets, contributing to improved accuracy	Thaseen and Kumar (2013)
	SMO classifier, the detection rate is 97%.	The work also uses SVM based classifier and achieves detection rate similar to (Chen et al., 2005).	Adebowale, et al. (2013)
	Expectation Maximization (EM) clustering, the accuracy is 78%	EM forms a “soft” task of each row to various clusters in percentage to the probability of each cluster. The accuracy in this method is low as EM does not give a parameter covariance matrix for standard errors	Ahmed, et al. (2016)
ADFA-WD	Creech et al. have used Hidden Markov Model (HMM), Extreme Learning Machine (ELM) and SVM. They reported 74.3% accuracy for HMM, 98.57%	The ADFA-WD is a much new data set and contains new attacks. Therefore reported accuracy was not as good as for every machine learning technique when compared to the accuracy using legacy KDD98 data.	Creech and Hu (2014b)

Dataset	Result	Observations	Reference
	accuracy for ELM and 99.64% accuracy for SVM.	SVM has been reported to produce the highest accuracy.	
ADFA-LD	100% accuracy for using ELM using original semantic feature	New semantic features are applied. Therefore, ELM, are capable to use the new semantic feature easily and quickly by including amounts of semantic phrases.	Creech and Hu (2014b)
CICIDS2017	94.5% accuracy obtained by using MLP solely, by using MLP and Payload Classifier together 95.2% accuracy rate is detected.	Feature selection is done by using Fisher Score algorithm.	Usteba, et al. (2018)
Bot-IoT	The highest accuracy from the SVM model. 98% detection rate	This SVM based method has produced good detection accuracy (Mitchell & Chen, 2015; Chen et al., 2005; Ferrari & Cribari-Neto, 2004)	Koroniotis, et al. (2018)

Fig 1.2.2 Summary of the datasets

1.3 TECHNOLOGIES

1.3.1 PYTHON

Python was the language of choice for this project. This was an easy decision for multiple reasons.

- Python as a language has an enormous community behind it. Any problems that might be encountered can be easily solved with a trip to Stack Overflow. Python is among the most popular languages on the site which makes it very likely there will be a direct answer to any query.
- Python has an abundance of powerful tools ready for scientific computing.
- Packages such as Numpy, Pandas, and SciPy are freely available and well documented. Packages such as these can dramatically reduce, and simplify the code needed to write a given program. This makes iteration quick.

- Python as a language is forgiving and allows for programs that look like pseudo code. This is useful when pseudocode given in academic papers needs to be implemented and tested. Using Python, this step is usually reasonably trivial.

However, Python is not without its flaws. The language is dynamically typed, and packages are notorious for Duck Typing. This can be frustrating when a package method returns something that, for example, looks like an array rather than being an actual array. Coupled with the fact that standard Python documentation does not explicitly state the return type of a method, this can lead to a lot of trials and error testing that would not otherwise happen in a strongly typed language. This is an issue that makes learning to use a new Python package or library more difficult than it otherwise could be.

1.3.2 NUMPY

Numpy is python modules which provide scientific and higher-level mathematical abstractions wrapped in python. In most of the programming languages, we cannot use mathematical abstractions such as f(x) as it would affect the semantics and the syntax of the code. But by using Numpy we can exploit such functions in our code. NumPy's array type augments the Python language with an efficient data structure used for numerical work, e.g., manipulating matrices. Numpy also provides basic numerical routines, such as tools for finding Eigenvectors.

1.3.3 SCIKIT LEARN

Scikit-learn is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support vector machine, random forest, gradient boosting, k-means etc. It is mainly designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy. Scikit-learn is largely written in Python, with some core algorithms written in Cython to achieve performance.

1.3.4 PANDAS

Pandas is a software library written for the Python programming language for data manipulation and analysis. It offers data structures and operations for manipulating numerical tables and time series. It is free software released under the three-clause BSD

license. The name is derived from the term "panel data", an econometrics term for data sets that include observations over multiple time periods for the same individuals. Its name is a play on the phrase "Python data analysis" itself.

The features of this library are:

- Data Frame object for data manipulation with integrated indexing
- Data filtration
- Tools for reading and writing data between in-memory data structures and different file formats
- Data structure column insertion and deletion.
- Data alignment and integrated handling of missing data.
- Reshaping, pivoting, merging, and joining of data sets.
- Label-based slicing, fancy indexing, and sub setting of large data sets.
- Group by engine allowing split-apply-combine operations on data sets.
- Hierarchical axis indexing to work with high-dimensional data in a lower-dimensional data structure.
- Time series-functionality: Date range generation and frequency conversion, moving window statistics, moving window linear regressions, date shifting and lagging.

The library is highly optimized for performance, with critical code paths written in Cython or C.

1.3.5 PICKLE

Python pickle module is used for serializing and de-serializing a Python object structure. Any object in Python can be pickled so that it can be saved on disk. What pickle does is that it “serializes” the object first before writing it to file. Pickling is a way to convert a python object (list, dictionary, etc.) into a character stream. The idea is that this character stream contains all the information necessary to reconstruct the object in another python script.

1.4 SYSTEM REQUIREMENTS

Software Requirements:

Windows or linux operating system with any python IDE supporting python 3.7 coding format.

Hardware Requirements:

I5 or greater processor with minimum 500 GB hard disk and minimum 16 GB RAM/Memory along with monitor size of 15 inch

Chapter 2

LITERATURE SURVEY

The following papers were studied to get an overview of the techniques that were applied earlier in intrusion detection.

CONVOLUTIONAL NEURAL NETWORKS WITH LSTM FOR INTRUSION DETECTION

-Mostofa Ahsan, Kendall E. Nygard, Department of Computer Science, North Dakota State University

A variety of attacks are regularly attempted at network infrastructure. With the increasing development of artificial intelligence algorithms, it has become effective to prevent network intrusion for more than two decades. Deep learning methods can achieve high accuracy with a low false alarm rate to detect network intrusions. A novel approach using a hybrid algorithm of Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) is introduced in this paper to provide improved intrusion detection. This bidirectional algorithm showed the highest known accuracy of 99.70% on a standard dataset known as NSL KDD. The performance of this algorithm is measured using precision, false positive, F1 score, and recall which found promising for deployment on live network infrastructure.

NETWORK INTRUSION DETECTION SYSTEM

-Gopalkrishna N. Prabhu, Kushank Jain, Nandan Lawande, Narendra Kumar, Yashasvi Zutshi, Rohan Singh, Jyoti Chinchole, Department of Information Technology, Atharva College of Engineering, University of Mumbai, India

Attacks on computers and data networks have become a regular and sophisticated issue. Intrusion detection has shifted its attention from hosts and operating systems to networks and has become a way to provide a sense of security to these networks. The aim of intrusion detection is to detect misuse and unauthorized use of the computer systems by internal and external elements. Typically, Intrusion Detection Systems allow statistical anomaly and rule-based misuse models to detect intrusions as the behaviour of the intruding element is considered to be different from the authorized user behaviour.

TOWARD GENERATING A NEW INTRUSION DETECTION DATASET AND INTRUSION TRAFFIC CHARACTERIZATION

-Iman Sharafaldin, Arash Habibi Lashkari and Ali A. Ghorbani, Canadian Institute for Cybersecurity (CIC), University of New Brunswick (UNB), Canada

With exponential growth in the size of computer networks and developed applications, the significant increasing of the potential damage that can be caused by launching attacks is becoming obvious. Meanwhile, Intrusion Detection Systems (IDSs) and Intrusion Prevention Systems (IPSs) are one of the most important defence tools against the sophisticated and ever-growing network attacks. Due to the lack of adequate dataset, anomaly-based approaches in intrusion detection systems are suffering from accurate deployment, analysis and evaluation. There exist a number of such datasets such as DARPA98, KDD99, ISC2012, and ADFA13 that have been used by the researchers to evaluate the performance of their proposed intrusion detection and intrusion prevention approaches. Based on our study over eleven available datasets since 1998, many such datasets are out of date and unreliable to use. Some of these datasets suffer from lack of traffic diversity and volumes, some of them do not cover the variety of attacks, while others anonymized packet information and payload which cannot reflect the current trends, or they lack feature set and metadata. This paper produces a reliable dataset that contains benign and seven common attack network flows, which meets real world criteria and is publicly available. Consequently, the paper evaluates the performance of a comprehensive set of network traffic features and machine learning algorithms to indicate the best set of features for detecting the certain attack categories.

INTRUSION DETECTION SYSTEM

-Mr Mohit Tiwari, Raj Kumar, Akash Bharti, Jai Kishan, Department of CSE, Bharati Vidyapeeth's College of Engineering, New Delhi, India

Intrusion Detection System (IDS) defined as a Device or software application which monitors the network or system activities and finds if there is any malicious activity occur. Outstanding growth and usage of internet raises concerns about how to communicate and protect the digital information safely. In today’s world hackers use different types of attacks for getting the valuable information. Many of the intrusion detection techniques, methods and algorithms help to detect those several attacks. The main objective of this paper is to provide a complete study about the intrusion detection,

types of intrusion detection methods, types of attacks, different tools and techniques, research needs, challenges and finally develop the IDS Tool for Research Purpose That tool are capable of detect and prevent the intrusion from the intruder.

ENSEMBLE CLASSIFIERS FOR NETWORK INTRUSION DETECTION SYSTEM

-Anazida Zainal, Mohd Aizaini Maarof, Siti Mariyam Shamsuddin, Universti Teknologi Malaysia, 81310 Skudai, Johor, Malaysia

Two of the major challenges in designing anomaly intrusion detection are to maximize detection accuracy and to minimize false alarm rate. In addressing this issue, this paper proposes an ensemble of one-class classifiers where each adopts different learning paradigms. The techniques deployed in this ensemble model are; Linear Genetic Programming (LGP), Adaptive Neural Fuzzy Inference System (ANFIS) and Random Forest (RF). The strengths from the individual models were evaluated and ensemble rule was formulated. Prior to classification, a 2-tier feature selection process was performed to expedite the detection process. Empirical results show an improvement in detection accuracy for all classes of network traffic; Normal, Probe, DoS, U2R and R2L. Random Forest, which is an ensemble learning technique that generates many classification trees and aggregates the individual result was also able to address imbalance dataset problem that many of machine learning techniques fail to sufficiently address it.

AN INTRODUCTION TO INTRUSION-DETECTION SYSTEMS

-Herve Debar, IBM Research, Zurich Research Laboratory, Switzerland

Intrusion-detection systems aim at detecting attacks against computer systems and networks or, in general, against information systems. Indeed, it is difficult to provide provably secure information systems and to maintain them in such a secure state during their lifetime and utilization. Sometimes, legacy, or operational constraints do not even allow the definition of a fully secure information system. Therefore, intrusion detection systems have the task of monitoring the usage of such systems to detect any apparition of insecure states. They detect attempts and active misuse either by legitimate users of the information systems or by external parties to abuse their privileges or exploit security vulnerabilities. This paper is the first in a two-part series; it introduces the concepts used in intrusion detection systems around a taxonomy.

INTRUSION DETECTION SYSTEM (IDS): ANOMALY DETECTION USING OUTLIER DETECTION APPROACH

-Jabez Ja, Dr.B.Muthukumar, Sathyabama University, Sholinganallur, Chennai

An Intrusion Detection System (IDS) is a software application or device that monitors the system or activities of network for policy violations or malicious activities and generates reports to the management system. A number of systems may try to prevent an intrusion attempt, but this is neither required nor expected of a monitoring system. The main focus of Intrusion detection and prevention systems (IDPS) is to identify the possible incidents, logging information about them and in report attempts. In addition, organizations use IDPS for other purposes, like identifying problems with security policies, deterring individuals and documenting existing threats from infringing security policies. IDPS have become an essential addition to the security infrastructure of nearly every organization. Various methods can be used to detect intrusions but each one is specific to a specific method. The main goal of an intrusion detection system is to detect the attacks efficiently. Furthermore, it is equally important to detect attacks at a beginning stage in order to reduce their impacts. This research work proposed a new approach called outlier detection where, the anomaly dataset is measured by the Neighbourhood Outlier Factor (NOF). Here, trained model consists of big datasets with distributed storage environment for improving the performance of Intrusion Detection system. The experimental results proved that the proposed approach identifies the anomalies very effectively than any other approaches.

MACHINE LEARNING BASED INTRUSION DETECTION SYSTEM

-Anish Halimaa A, Dr. K. Sundarakantham, Department of Computer Science and Engineering, Thiagarajar College of Engineering, Madurai, India

In order to examine malicious activity that occurs in a network or a system, intrusion detection system is used. Intrusion Detection is software or a device that scans a system or a network for a distrustful activity. Due to the growing connectivity between computers, intrusion detection becomes vital to perform network security.

Various machine learning techniques and statistical methodologies have been used to build different types of Intrusion Detection Systems to protect the networks. Performance of an Intrusion Detection is mainly depends on accuracy. Accuracy for Intrusion detection must be enhanced to reduce false alarms and to increase the

detection rate. In order to improve the performance, different techniques have been used in recent works. Analyzing huge network traffic data is the main
work of intrusion detection system. A well-organized classification methodology is required to overcome this issue. This issue is taken in proposed approach. Machine learning techniques like Support Vector Machine (SVM) and Naïve Bayes are applied. These techniques are well-known to solve the classification problems. For evaluation of intrusion detection system, NSL– KDD knowledge discovery Dataset is taken. The outcomes show that SVM works better than Naïve Bayes. To perform comparative analysis, effective classification methods like Support Vector Machine and Naive Bayes are taken, their accuracy and misclassification rate get calculated.

NETWORK INTRUSION DETECTION SYSTEM USING VOTING ENSEMBLE MACHINE LEARNING

-Md. Raihan-Al-Masud, Hossen Asiful Mustafa, Institute of Information and Communication Technology, Bangladesh University of Engineering and Technology Palashi, Dhaka-1205, Bangladesh

Due to increasing amount of cyber-attack, there is a growing demand for Network intrusion detection systems (NIDSs) which are necessary for defending from potential attacks. Detecting and preventing cyber-attacks is one of the key research areas. Existing NIDSs use traditional machine learning algorithms with low accuracy and are also not suitable for the new unknown cyber-attacks. In this paper, we propose a NIDS model with ensemble machine learning methods. Ensemble machine learning methods have the potential to detect and prevent different types of attacks compared to traditional machine learning methods. Our proposed system can detect known attacks as well as can prevent unknown attacks. Our proposed system uses ensemble machine learning methods with Voting. We used the full NSL-KDD dataset to evaluate the performance of multiclass classification and we also compare the performance with deep learning as well as traditional base level machine learning techniques. Experimental results show that the proposed NIDS system is superior to the performance of existing methods. Our model improves the detection rate of the IDS which is vital for network intrusion detection systems.

Chapter 3

EXISTING SYSTEM & PROPOSED SYSTEM

3.1 Existing System

The network intrusion detection is implemented mostly using anomaly-based techniques which are machine learning based approaches that identify patterns in network anomaly dataset which contains different scenarios of attacks. One of the approaches applied in anomaly-based techniques is Support Vector Machines.

In machine learning, support vector machines (SVMs, also support-vector networks) are supervised learning models with associated learning algorithms that analyse data for classification and regression. Developed at AT&T Bell Laboratories by Vladimir Vapnik with colleagues (Boser et al., 1992, Guyon et al., 1993, Vapnik et al., 1997), SVMs are one of the most robust prediction methods, being based on statistical learning frameworks or VC theory proposed by Vapnik (1982, 1995) and Chervonenkis (1974). Given a set of training examples, each marked as belonging to one of two categories, an SVM training algorithm builds a model that assigns new examples to one category or the other, making it a non-probabilistic binary linear classifier (although methods such as Platt scaling exist to use SVM in a probabilistic classification setting). SVM maps training examples to points in space to maximise the width of the gap between the two categories. New examples are then mapped into that same space and predicted to belong to a category based on which side of the gap they fall.

In addition to performing linear classification, SVMs can efficiently perform a non-linear classification using what is called the kernel trick, implicitly mapping their inputs into high-dimensional feature spaces.

In a high dimensional space, SVM creates hyperplane or multiple hyperplanes to categorize or plot the data points. In this algorithm, we plot each data item as a point in n-dimensional space (where n is number of features you have) with the value of each feature being the value of a particular coordinate.
This method when applied on a network dataset like NSL-KDD it shows accuracy between 93 to 98 percent.

The main disadvantages of SVM for this kind of data is:

- Model is less stable in testing phase.
- Model training is slow.
- There is a high risk of training getting into a deadlock or hanged situation if resources are not correctly distributed or if the dataset is too large even after pre-processing.

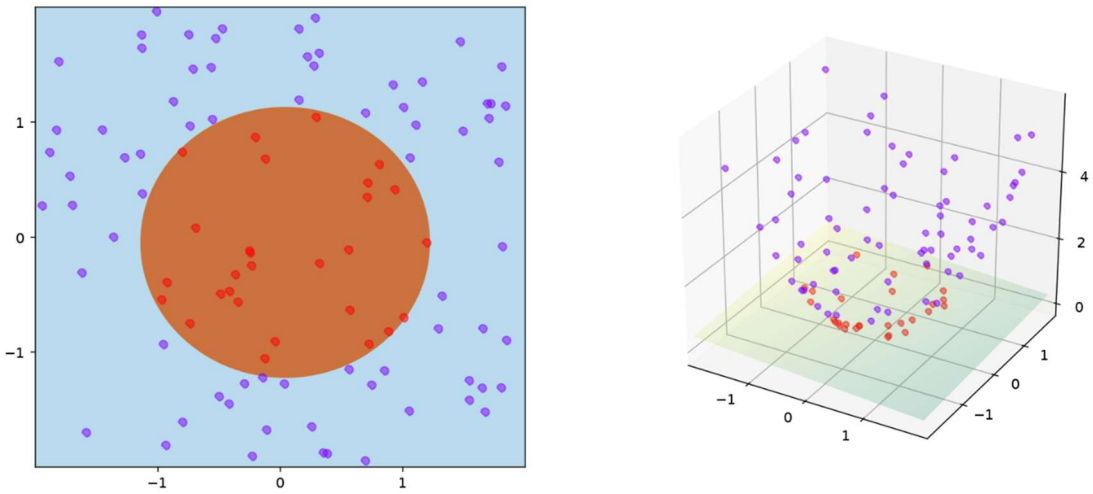


Fig 3.1.1 A training example of SVM

3.2 Proposed System

We propose to use an ensemble-based classifier which can incorporate power of multiple models into a single model. An ensemble method is a technique which uses multiple independent similar or different models/weak learners to derive an output or make some predictions. For e.g. A random forest is an ensemble of multiple decision trees. An ensemble can also be built with a combination of different models like random forest, SVM, Logistic regression etc.

The ensemble learning category contains many algorithms and has a facility to create new kind of ensemble using existing algorithms with help of classifier algorithms like voting classifier. We have selected following three algorithms for our training purpose:

- Xgboost
- Random Forest
- Extra Trees

We have also included special ensembles of above algorithms using voting classifier.

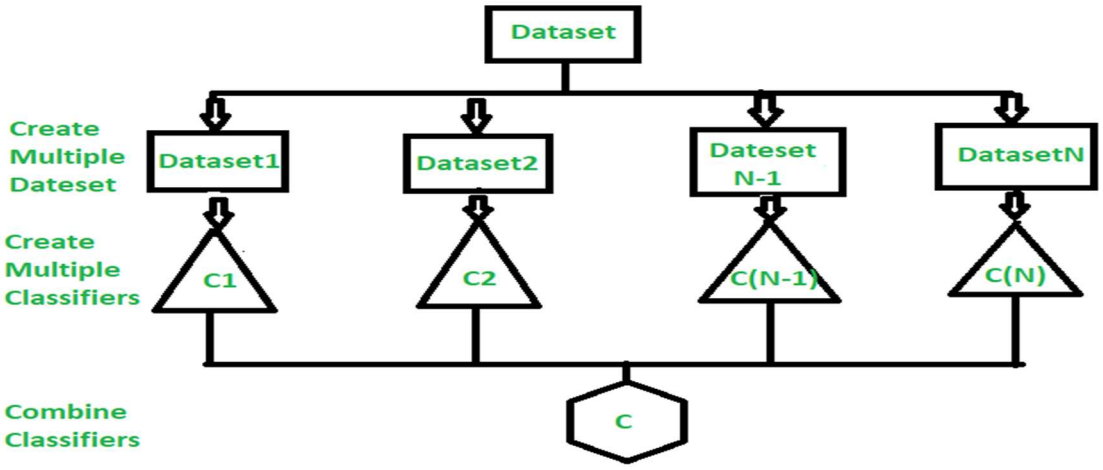


Fig 3.2.1 Ensemble Learning

We have also included special ensembles of above algorithms using voting classifier.

Xgboost

Xgboost is a decision-tree-based ensemble Machine Learning algorithm that uses a gradient boosting framework**.** It improves upon the base GBM framework through systems optimization and algorithmic enhancements.

Random Forest

A random forest is a meta estimator that fits several decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting.

Extra Trees

The Extra-Tree method (standing for extremely randomized trees) was proposed with the main objective of further randomizing tree building in the context of numerical input features, where the choice of the optimal cut-point is responsible for a large proportion of the variance of the induced tree.

This method consists of following advantages:

- This method allows for much more flexible structure to exist.
- This method gives better accuracy, more stable and higher computation rate.

Chapter 4

ALGORITHMS

Accuracy is the main concern of most of the intrusion detection system. With a wide variety of intrusions happening, it becomes difficult to identify the intrusion. An intrusion may have a different pattern or a different behaviour. Hence, it is important to build a system that can effectively detect these kinds of intrusions. The existing systems are based on single algorithm approach such as support vector machines (SVM), naïve bayes classifier etc. These kinds of algorithms follow a particular pattern or have a limited set of attributes to detect the intrusion. They fail when the intrusion does not fall into the patterns followed by them. Hence, to improve the detection rate, we propose to use ensemble model.

4.1 Ensemble Classifier

Ensemble methods is a machine learning technique that combines several base models to produce one optimal predictive model. This approach allows the production of better predictive performance compared to a single model. Basic idea behind ensemble classifiers is to learn a set of classifiers and to allow them to vote.

In general, an ensemble model falls into one of these two categories:

1. Sequential approach
2. Parallel approach

4.1.1 Sequential approach

A sequential ensemble model operates by having the base learners/models generated in sequence. Sequential ensemble methods are typically used to try and increase overall performance, as the ensemble model can compensate for inaccurate predictions by re-weighting the examples that were previously misclassified. Example: Adaboost

4.1.2 Parallel approach

A parallel model consists of methods that rely on creating and training the base learners in parallel. Parallel methods aim to reduce the error rate by training many models in parallel and averaging the results together. Example: Random Forest Classifier

The ensemble learning category contains many algorithms and has a facility to create new kind of ensemble using existing algorithms with help of classifier algorithms like voting classifier.

An Ensemble model can be implemented using 2 methods namely bagging and boosting

4.1.3 Bagging

Bagging is a classification method that aims to reduce the variance of estimates by averaging multiple estimates together. Bagging creates subsets from the main dataset that the learners are trained on.

In order for the predictions of the different classifiers to be aggregated, either an averaging is used for regression, or a voting approach is used for classification (based on the decision of the majority).

One example of a bagging classification method is the *Random Forests Classifier*. In the case of the random forest classifier, all the individual trees are trained on a different sample of the dataset.

The tree is also trained using random selections of features. When the results are averaged together, the overall variance decreases and the model performs better as a result.

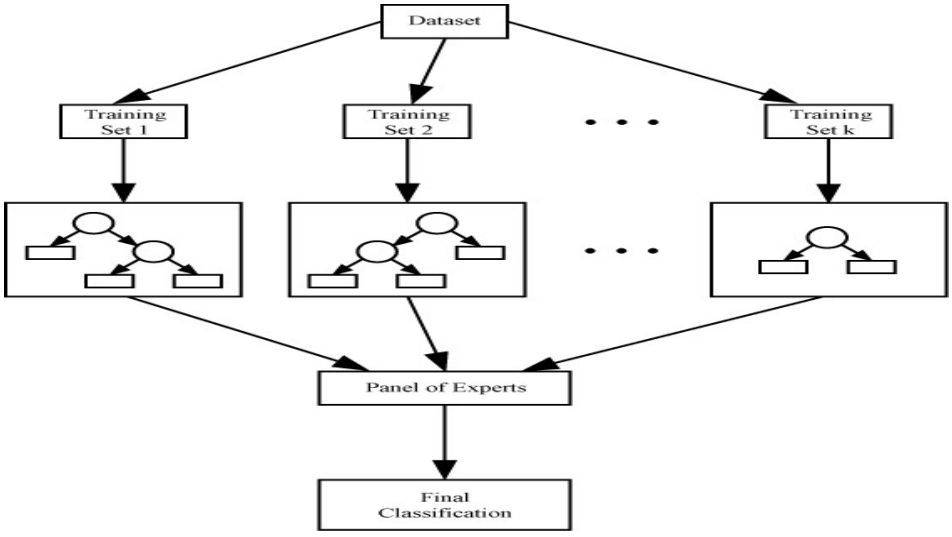


Fig 4.1 Bagging classifier

4.1.4 Boosting

Boosting algorithms can take weak, underperforming models and converting them into strong models. The idea behind boosting algorithms is that you assign many weak learning models to the datasets, and then the weights for misclassified examples are tweaked during subsequent rounds of learning.

The predictions of the classifiers are aggregated and then the final predictions are made through a weighted sum (in the case of regressions), or a weighted majority vote (in the case of classification).

One example of boosting is Adaboost classifier.

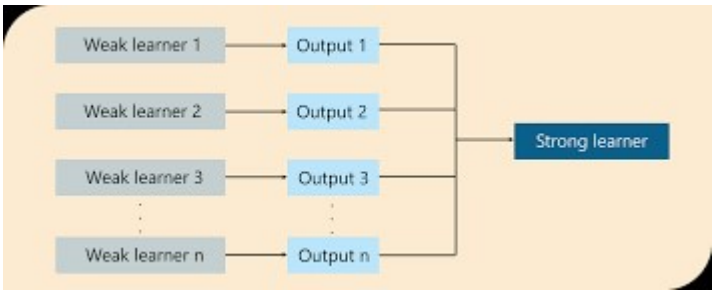


Fig 4.2 Boosting classifier

4.1.5 Voting Classifier

A Voting Classifier is a machine learning model that trains on an ensemble of numerous models and predicts an output (class) based on their highest probability of chosen class as the output.

Voting Classifier supports two types of voting:

- **Hard Voting:** In hard voting, the predicted output class is a class with the highest majority of votes i.e. the class which had the highest probability of being predicted by each of the classifiers
- **Soft Voting**: In soft voting, the output class is the prediction based on the average of probability given to that class.

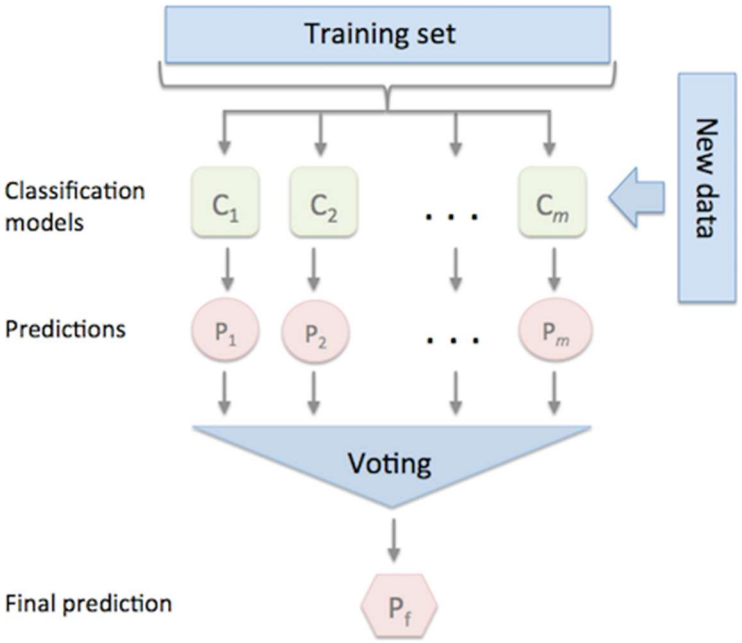


Fig 4.3 Voting Classifier

4.2 Xgboost

XGBoost is a decision-tree-based ensemble algorithm that uses a gradient boosting framework. XGBoost and Gradient Boosting Machines (GBMs) are both ensemble tree methods that apply the principle of boosting weak learners using the gradient descent architecture. However, XGBoost improves upon the base GBM framework through systems optimization and algorithmic enhancements.

The beauty of this powerful algorithm lies in its scalability, which drives fast learning through parallel and distributed computing and offers efficient memory usage. XGBoost is a popular implementation of gradient boosting. Some features of XGBoost that make it so interesting are:

- **Regularization:** XGBoost has an option to penalize complex models through both L1 and L2 regularization. Regularization helps in preventing overfitting.
- **Handling sparse data:** Missing values or data processing steps like one-hot encoding make data sparse. XGBoost incorporates a sparsity-aware split finding algorithm to handle different types of sparsity patterns in the data.
- **Weighted quantile sketch:** Most existing tree-based algorithms can find the split points when the data points are of equal weights (using quantile sketch algorithm). However, they are not equipped to handle weighted data. XGBoost

has a distributed weighted quantile sketch algorithm to effectively handle weighted data.

- **Block structure for parallel learning:** For faster computing, XGBoost can make use of multiple cores on the CPU. This is possible because of a block structure in its system design. Data is sorted and stored in in-memory units called blocks. Unlike other algorithms, this enables the data layout to be reused by subsequent iterations, instead of computing it again. This feature also serves useful for steps like split finding and column sub-sampling
- **Cache awareness:** In XGBoost, non-continuous memory access is required to get the gradient statistics by row index. Hence, XGBoost has been designed to make optimal use of hardware. This is done by allocating internal buffers in each thread, where the gradient statistics can be stored
- **Out-of-core computing:** This feature optimizes the available disk space and maximizes its usage when handling huge datasets that do not fit into memory.

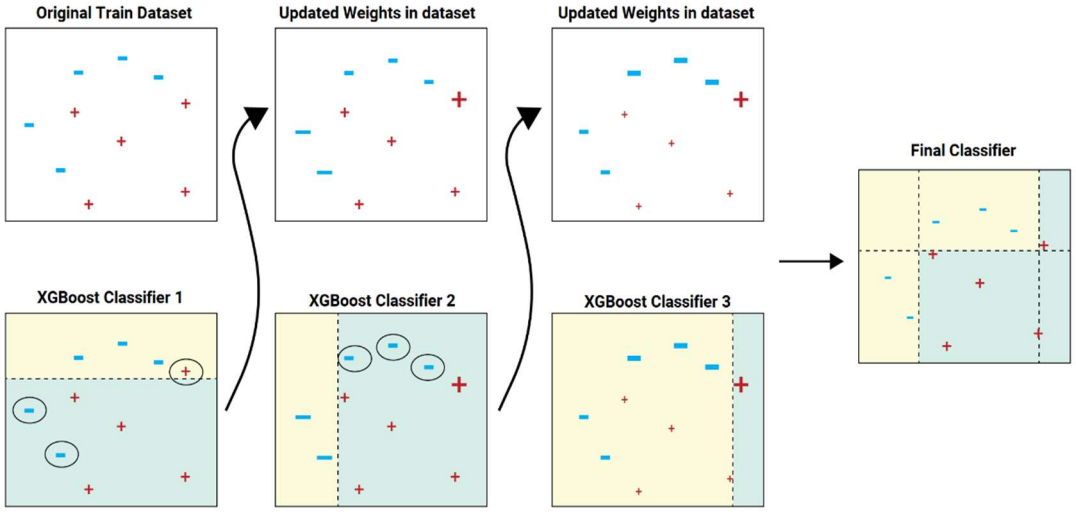


Fig 4.4 Xgboost classifier

4.3 Random Forest Classifier

Random forest, like its name implies, consists of a large number of individual decision trees that operate as an ensemble. It uses bagging and feature randomness when building each individual tree to try to create an uncorrelated forest of trees whose prediction by committee is more accurate than that of any individual tree. Each individual tree in the random forest spits out a class prediction and the class with the most votes becomes our model's prediction.

The fundamental concept behind random forest is a simple but powerful one i.e; A large number of relatively uncorrelated models (trees) operating as a committee will outperform any of the individual constituent models. The low correlation between models is the key.

One of the biggest advantages of random forest is its versatility. It can be used for both regression and classification tasks, and it's also easy to view the relative importance it assigns to the input features.

Another great quality of the random forest algorithm is that it is very easy to measure the relative importance of each feature on the prediction.

Random forest is also a very handy algorithm because the default hyperparameters it uses often produce a good prediction result.

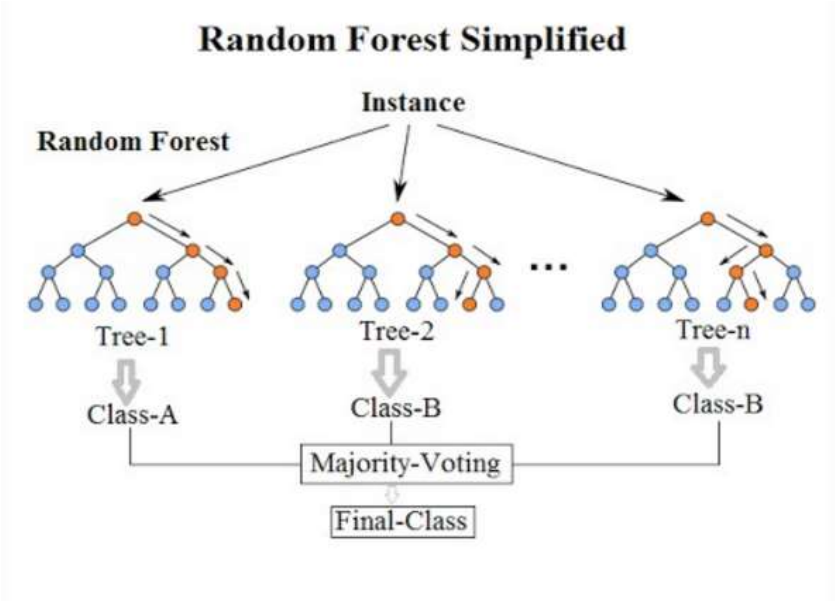


Fig 4.5 Random Forest Classifier

The random forest algorithm is used in a lot of different fields, like banking, the stock market, medicine and e-commerce.

Random forest is a great algorithm to train early in the model development process, to see how it performs. The algorithm is also a great choice for anyone who needs to develop a model quickly. On top of that, it provides a pretty good indicator of the importance it assigns to your features.

4.4 Extra Trees Classifier

Extremely Randomized Trees Classifier (Extra Trees Classifier) is a type of ensemble learning technique which aggregates the results of multiple de-correlated decision trees collected in a "forest" to output its classification result. In concept, it is very similar to a Random Forest Classifier and only differs from it in the manner of construction of the decision trees in the forest.

The Extra Trees algorithm works by creating a large number of unpruned decision trees from the training dataset. Predictions are made by averaging the prediction of the decision trees in the case of regression or using majority voting in the case of classification.

Unlike bagging and random forest that develop each decision tree from a bootstrap sample of the training dataset, the Extra Trees algorithm fits each decision tree on the whole training dataset. The Extra-Trees algorithm builds an ensemble of unpruned decision or regression trees according to the classical top-down procedure. Its two main differences with other tree-based ensemble methods are that it splits nodes by choosing cut-points fully at random and that it uses the whole learning sample (rather than a bootstrap replica) to grow the trees. The random selection of split points makes the decision trees in the ensemble less correlated, although this increases the variance of the algorithm. This increase in variance can be countered by increasing the number of trees used in the ensemble.

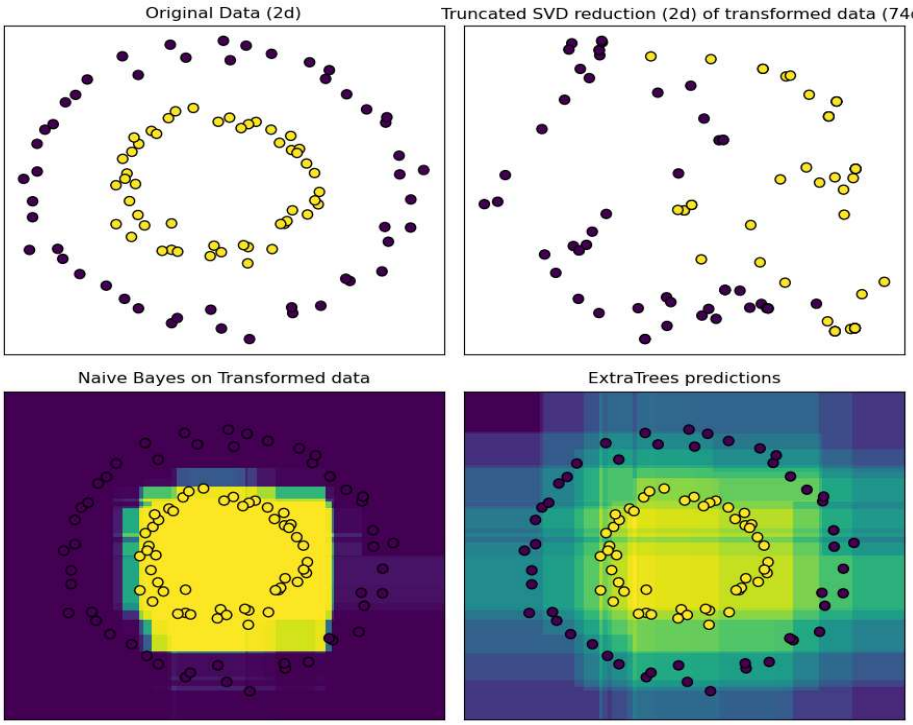


Fig 4.6 Extra Trees Classifier comparison

Chapter 5

SYSTEM DESIGN AND UML DESIGN

Design is concerned with identifying software components specifying relationships among components. Specifying structure and providing blueprint for the document phase. Modularity is one of the desirable properties of large systems. It implies that the system is divided into several parts. In such a manner, the interaction between parts is minimal clearly specified. Design will explain software components in detail. This will help the implementation of the system. Moreover, this will guide the further changes in the system to satisfy the future requirements. The purpose of the design phase is to plan a solution of the problem specified by the requirement document. This phase is the first step in moving from problem domain to the solution domain. The design of a system is perhaps the most critical factor affecting the quality of the software, and has a major impact on the later phases, particularly testing and maintenance. The output of this phase is the design document. This document is similar to a blueprint or plan for the solution, and is used later during implementation, testing and maintenance. The design activity is often divided into two separate phase system design and detail design. System design, which is sometimes also called top-level design, aims to identify the modules that should be in the system, the specifications of these modules, and how they interact with each other to produce the desired results. At the end of system design all the major data structures, file formats, output formats, as well as the major modules in the system and their specification are decided.

During detailed design the internal logic of the modules specified in system design is decided. During this phase further details of the data structures and algorithmic design of each of the modules is specified. The logic of a module is usually specified in a high-level design description language, which is independent of the target language in which the software will eventually be implemented. In the system design the focus is on identifying the modules, whereas during detailed design the focus is on designing the logic for each of the modules. In other words, in system design the attention is on what components are needed, while in detailed design how the components can be implemented in software is the issue. During the design phase, often two separate documents are produced, one for the system design and one for the detail design. Together, these documents completely specify the design of the system. That is, they specify the different in the system and internal logic of each of the modules.

A design methodology is a systematic approach to creating a design by application of set of techniques and guidelines. Most methodologies focus on system design. The two basic principles used in any design methodology are problem partitioning and abstraction. large systems cannot be handled as a whole, and so for design it is partitioned into smaller systems. Abstraction is a concept related to problem partitioning. When partitioning is used during design, the design activity focuses on one part of the system at a time. Since the part being designed interacts with other parts of the system, a clear understanding of the interaction is essential for properly designing the part. For this, abstraction is used. An abstraction of a system or a part defines the overall behaviour of the system at an abstract level without giving the internal details. While working with the part of a system, a designer needs to understand only the abstractions of the other parts with which the part being designed interacts. The use of abstraction allows the designer to practice the "divide and conquer" technique effectively by focusing one part at a time, without worrying about the details of other parts.

5.1 Unified Modelling Language (UML)

The Unified Modelling Language (UML) is a standard language for writing software blueprints. The creation of UML was originally motivated by the desire to standardize the disparate notational systems and approaches to software design. It was developed at Rational Software in 1994–1995, with further development led by them through 1996.

5.1.1 An Overview Of UML

The UML is the language for

- Visualizing
- Specifying
- Constructing
- Documenting

These are the artifacts of the software-intensive system.

5.1.2 A Conceptual Model of UML

The three major elements of UML are:

- The UML's basic building blocks
- The rules that dictate how those building blocks may be put together

- Some common mechanism that applies throughout the UML

5.1.3 Basic Building Blocks of the UML

The building blocks of UML can be defined as:

- Things
- Relationships
- Diagrams

Things are the abstractions that are first-class citizens in model. Relationships tie these things together. Diagrams group the interesting collection of things.

5.1.4 Things in the UML

Things are the most important building blocks of UML. Things can be:

- Structural things
- Behavioral things
- Grouping things
- Annotational things

5.1.5 Relationships in the UML

Relationship is another most important building block of UML. It shows how elements are associated with each other and this association describes the functionality of an application. There are four kinds of relationships available.

- Dependency
- Association
- Generalization
- Realization

Dependency:

Dependency is a relationship between two things in which change in one element also effects the other one.

Association:

Association is basically a set of links that connects elements of an UML model. It also describes how many objects are taking part in that relationship.

Generalization:

Generalization can be defined as a relationship which connects a specialized element with a generalized element. It basically describes inheritance relationship in the world of objects.

Realization:

Realization can be defined as a relationship in which two elements are connected. One element describes some responsibility which is not implemented and the other one implements them. This relationship exists in case of interfaces.

5.2 UML Diagrams

UML diagrams are the ultimate output of the entire discussion. All the elements, relationships are used to make a complete UML diagram and the diagram represents a system. The visual effect of the UML diagram is the most important part of the entire process. All the other elements are used to make it a complete one.

UML includes the following nine diagrams:

- Class diagram
- Object diagram
- Use case diagram
- Sequence diagram
- Collaboration diagram
- Activity diagram
- State chart diagram
- Deployment diagram
- Component diagram

5.2.1 Use Case Diagram

Use case diagrams are one of the five diagrams in the UML for modeling the dynamic aspects of systems (Activity diagrams, State chart diagrams, Sequence diagrams and Collaboration diagrams are four other diagrams for modeling dynamic aspects of systems).

A Use Case Diagram is a diagram that shows a set of Use Cases, Actors and their Relationships.

Contents Use case diagrams commonly contain:

- Use cases
- Actors

Dependency, generalization, and association relationships Like all other diagrams, use case diagrams may contain notes and constraints. Use case diagrams may also contain packages, which are used to group elements of your model into larger chunks. Use case diagrams are important for visualizing, specifying, and documenting the behavior of an element.

Use Case Diagram for Network Intrusion Detection

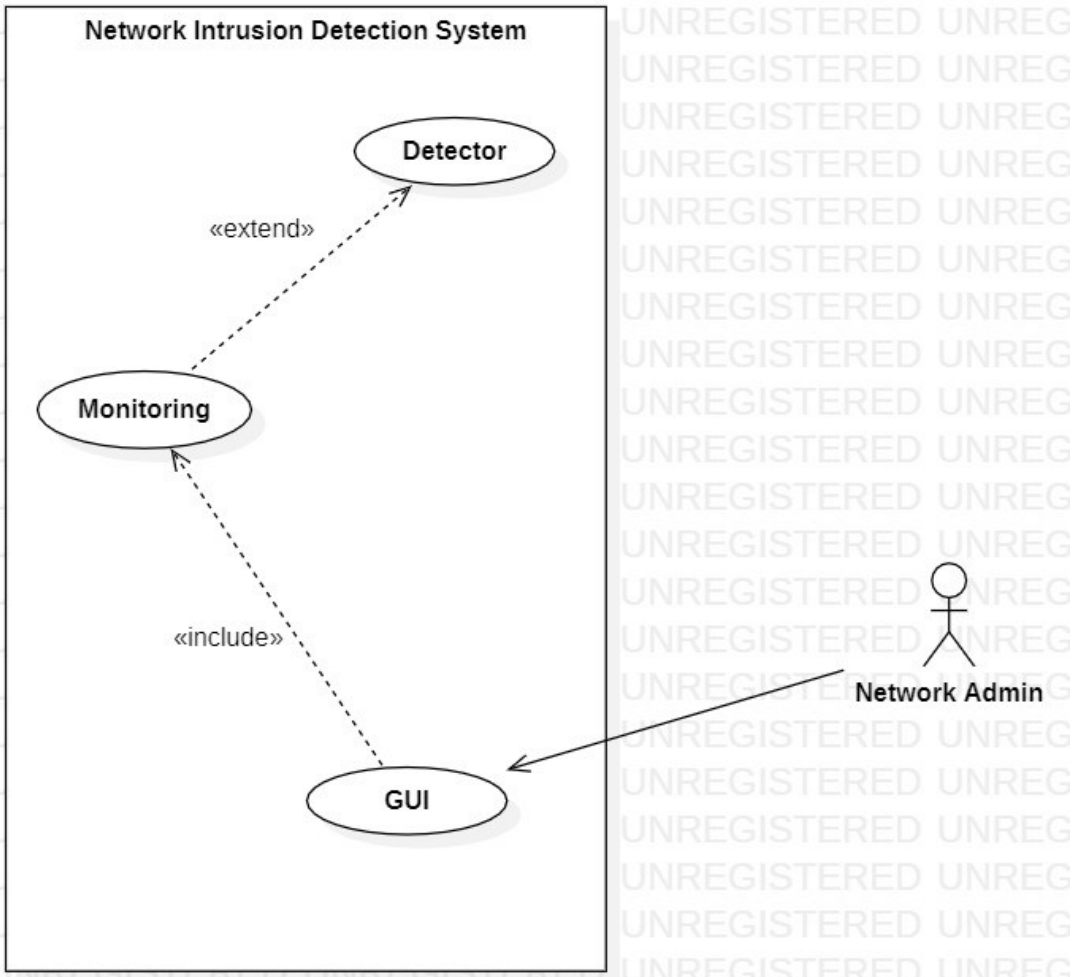


Fig 5.1 Use Case Diagram

5.2.2 Communication diagram

A communication diagram is the simplified version of collaboration diagram. It models the interactions between objects or parts in terms of sequenced messages. Communication diagrams represent a combination of information taken from Class, Sequence, and Use Case Diagrams describing both the static structure and dynamic behavior of a system.

However, communication diagrams use the free-form arrangement of objects and links as used in Object diagrams. To maintain the ordering of messages in such a free-form diagram, messages are labeled with a chronological number and placed near the link the message is sent over. Reading a communication diagram involves starting at message 1.0 and following the messages from object to object.

Communication diagrams show much of the same information as sequence diagrams, but because of how the information is presented, some of it is easier to find in one diagram than the other. Communication diagrams show which elements each one interacts with better, but sequence diagrams show the order in which the interactions take place more clearly.

Communication diagram corresponds (i.e., could be converted to/from or replaced by) to a simple sequence diagram without structuring mechanisms such as interaction uses and combined fragments.

Communication diagram consists of following parts:

- Frame
- Lifeline
- Message

Frames

Communication diagrams could be shown within a rectangular frame with the name in a compartment in the upper left corner.

Lifeline

Lifeline is a specialization of named element which represents an individual participant in the interaction. While parts and structural features may have multiplicity greater than 1, lifelines represent only one interacting entity

If the referenced connectable element is multivalued (i.e, has a multiplicity > 1), then the lifeline may have an expression that specifies which part is represented by this lifeline.

Message

Message in communication diagram is shown as a line with sequence expression and arrow above the line. The arrow indicates direction of the communication.

Communication Diagram for Model Formation And Training

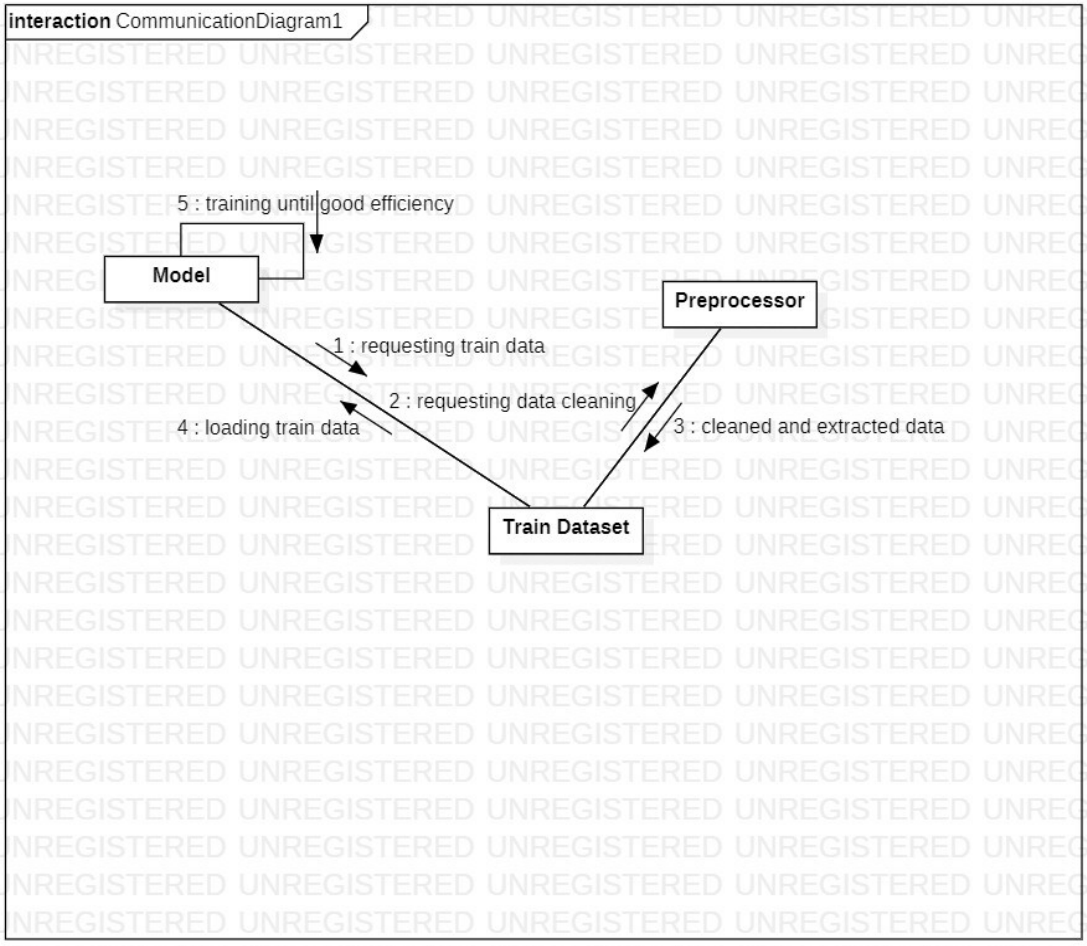


Fig 5.2 Communication diagram for training

Communication diagram for Detection

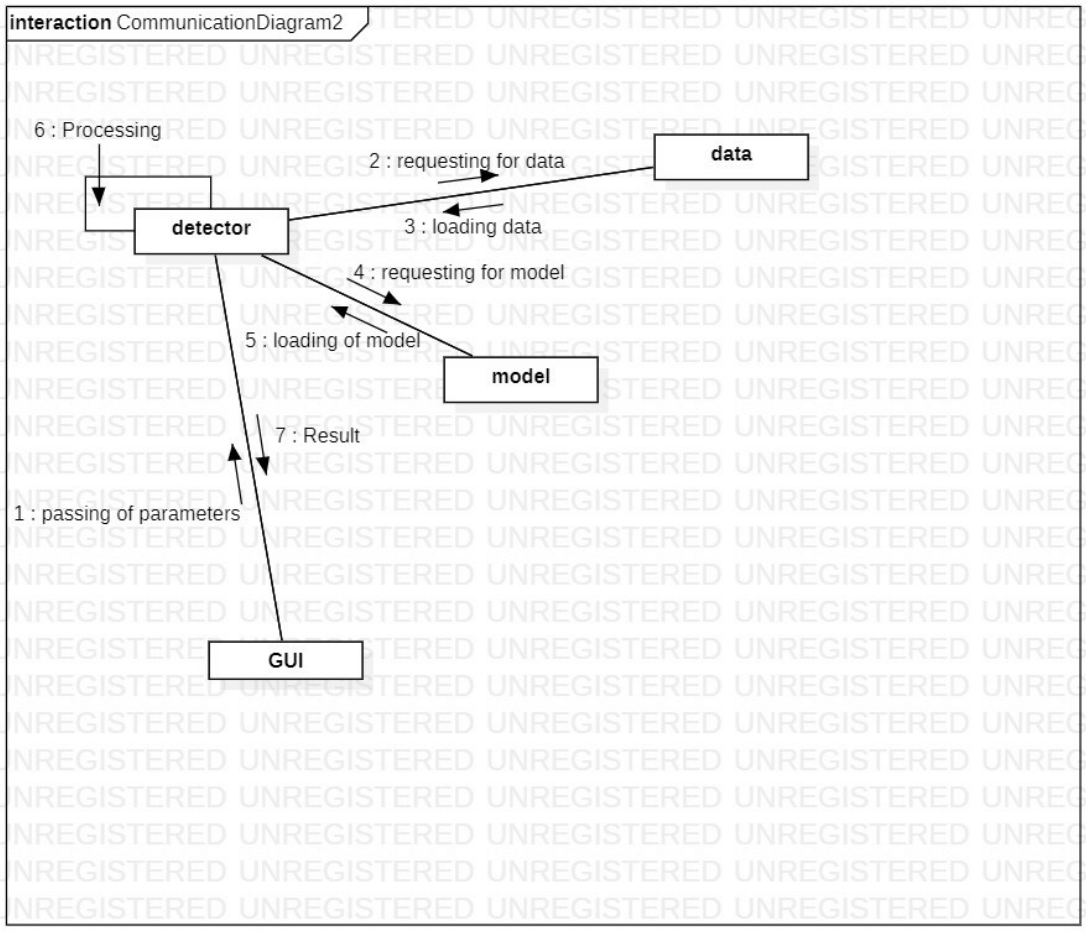


Fig 5.3 Communication diagram for Detection

5.2.3 Activity Diagram

Activity diagrams are one of the five diagrams in the UML for modeling the dynamic aspects of systems. (Use Case Diagrams, Sequence diagrams, Collaboration diagrams and State-chart diagrams are four other diagrams for modeling dynamic aspects of systems). An activity diagram is essentially a flowchart, showing flow of control from activity to activity. We use activity diagrams to model the dynamic aspects of a system. For the most part, this involves modeling the sequential (and possibly concurrent) steps in a computational process. With an activity diagram. We can also model the flow of an object as it moves from state to state at different points in the flow of control.

Terms and concepts:

1. An activity diagram shows the flow from activity to activity.

2. A transaction is an ongoing non-atomic execution within a state machine.
3. Activities ultimately result in some action, which is made up of executable atomic computations that result in a change in state of the system or the return of a value.
4. Actions encompass calling another operation, sending a signal, creating or destroying an object, or some pure computation, such as evaluating an expression.

Contents:

Activity diagrams commonly contain:

1. Action states and Activity states (Initial/Final State)
2. Transitions
3. Objects
4. Fork & Join
5. Branch
6. Swim lanes

Like all other diagrams, activity diagrams may contain notes and constraints.

Activity diagram for Model Training

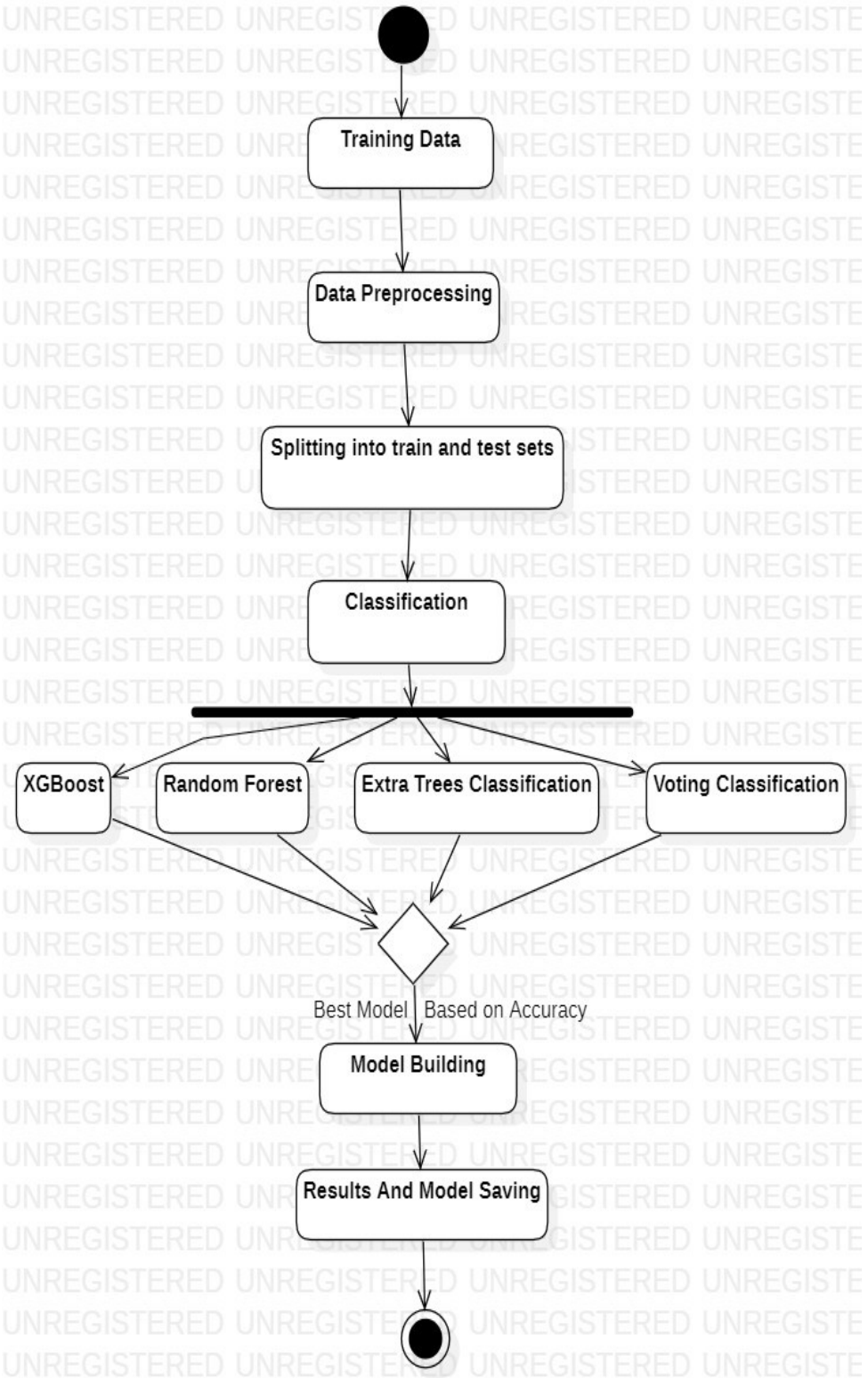


Fig 5.4 Activity diagram for model training

Activity Diagram for Detection

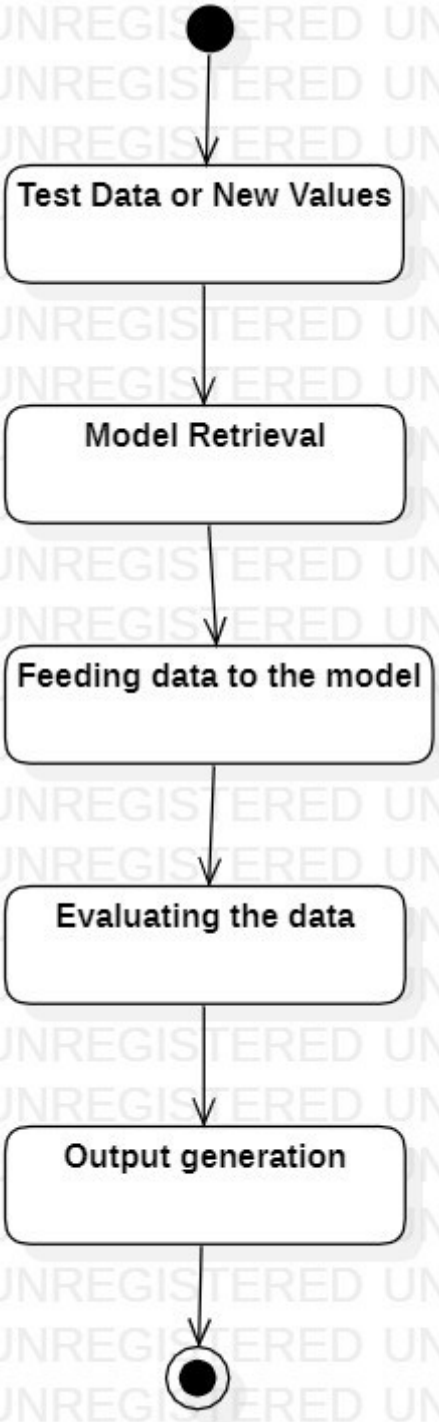


Fig 5.5 Activity Diagram for Detection

5.2.4 Sequence Diagram

Sequence Diagrams describe interactions among classes. These interactions are modelled as exchanges of messages. These diagrams focus on classes and the messages they exchange to accomplish some desired behaviour. Sequence diagrams are a type of interaction diagrams.

Sequence diagrams contain the following elements:

1. Class roles: which represent roles that objects may play within the interaction.
2. Lifelines: which represent the existence of an object over a period of time.
3. Activations: which represent the time during which an object is performing an operation.
4. Messages: which represent communication between objects.

Sequence diagram for Network Intrusion Detection

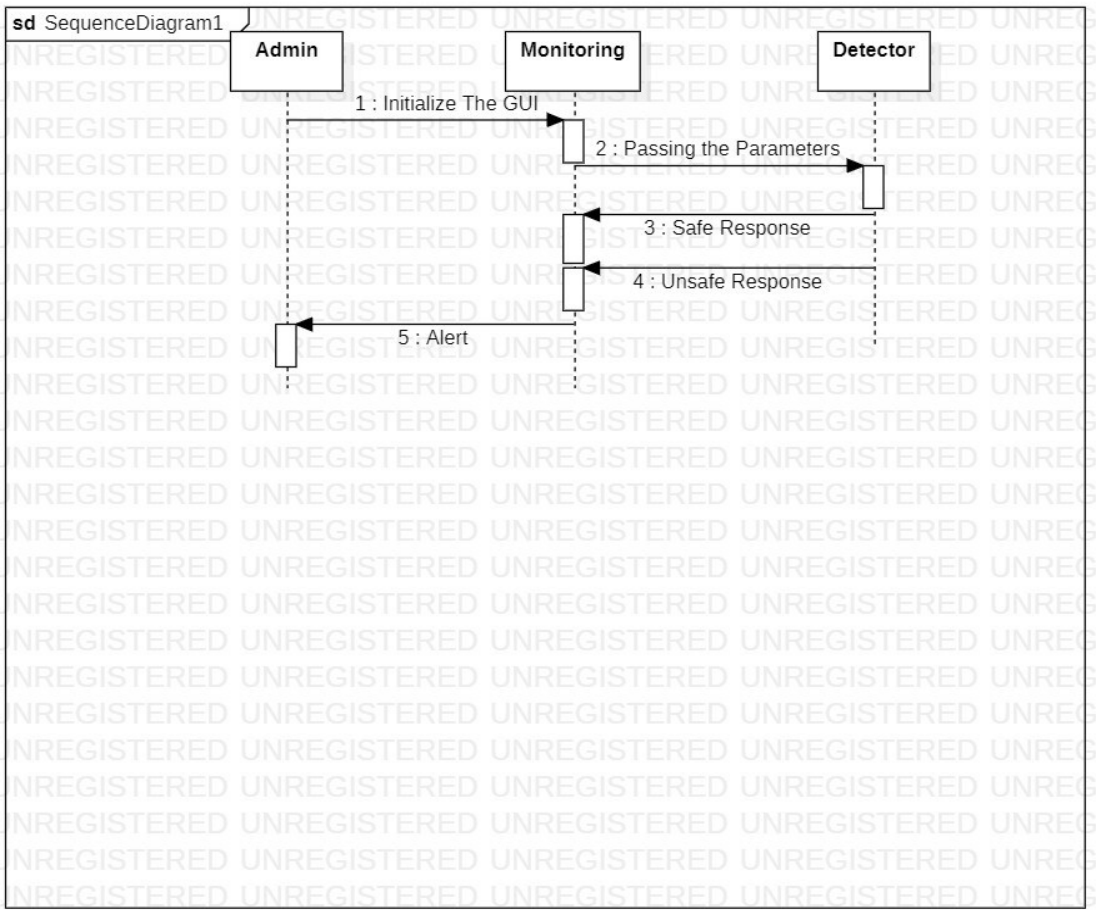


Fig 5.6 Sequence Diagram for Network Intrusion Detection

5.2.5 Class Diagram

Class Diagrams describe the static structure of a system, or how it is structured rather than how it behaves. A class diagram shows the existence of classes and their relationships in the logical view of a system.

The class diagram is the main building block of object-oriented modelling. It is used for general conceptual modelling of the structure of the application, and for detailed modelling, translating the models into programming code. Class diagrams can also be used for data modelling. The classes in a class diagram represent both the main elements, interactions in the application, and the classes to be programmed.

These diagrams contain the following elements:

- Classes and their structure and behaviour
- Association, aggregation, dependency, and inheritance relationships
- Multiplicity and navigation indicators
- Role names

In the diagram, classes are represented with boxes that contain three compartments:

- The top compartment contains the name of the class. It is printed in bold and centred, and the first letter is capitalized.
- The middle compartment contains the attributes of the class. They are left-aligned, and the first letter is lowercase.
- The bottom compartment contains the operations the class can execute. They are also left-aligned, and the first letter is lowercase.

In the design of a system, several classes are identified and grouped together in a class diagram that helps to determine the static relations between them. In detailed modelling, the classes of the conceptual design are often split into subclasses.

These diagrams are the most common diagrams found in O-O modelling systems.

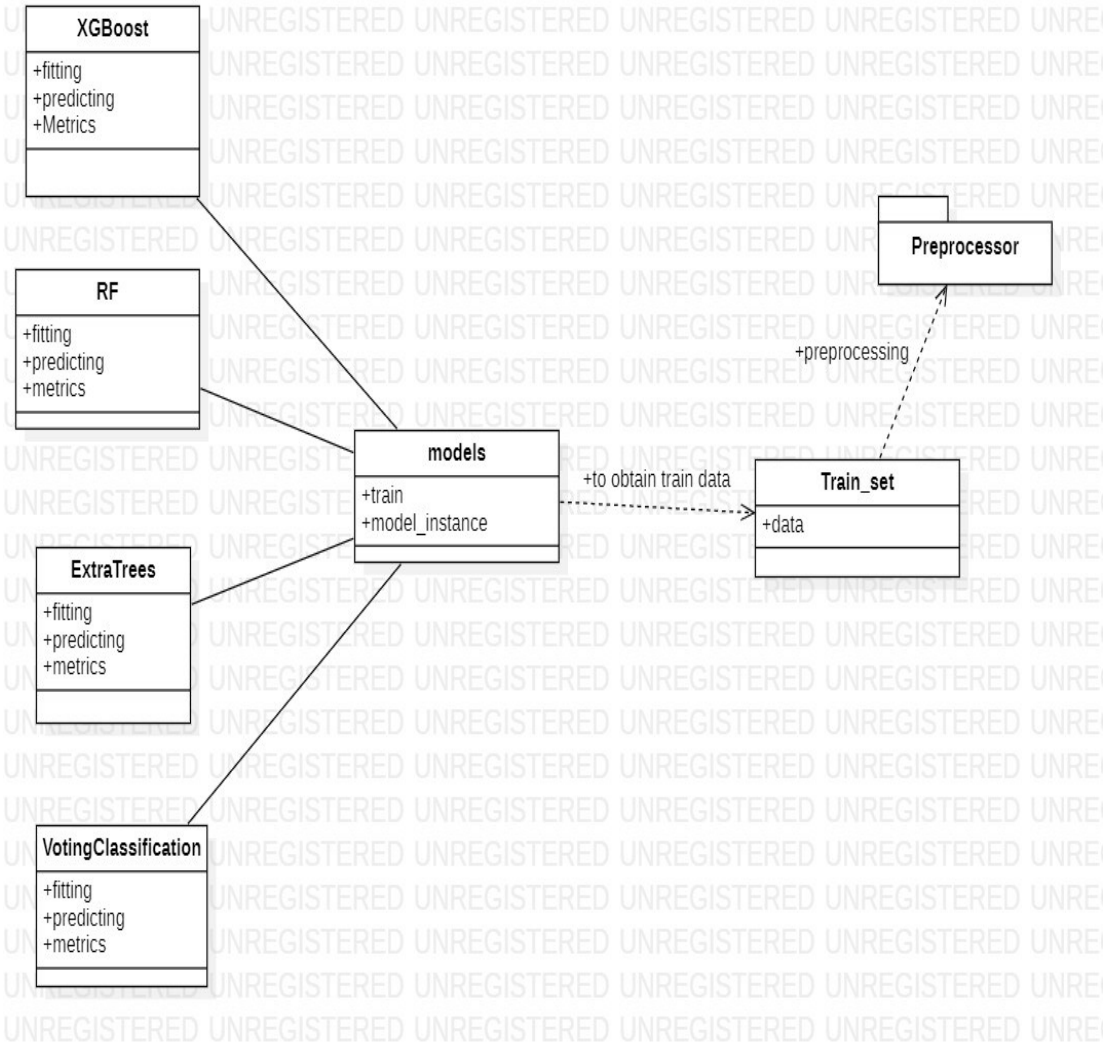


Fig 5.7 Class diagram for training mode

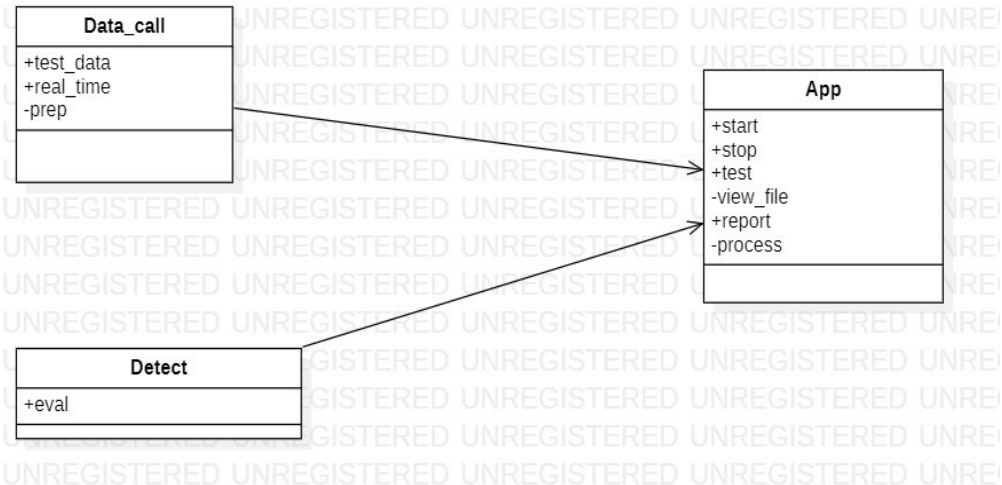


Fig 5.8 Class diagram for Detection

Chapter 6

SOURCE CODE

The Network Intrusion Detection code as following modules:

- Data
- Detector
- GUI
- Model Training
- Pre-processor
- Train data

6.1 Data Module

Data module is an important module that works in coordination with GUI. Its main task is to retrieve a particular data from database and send it to the GUI when the command is sent by the GUI to it.

The data module forms the base for the GUI. It can retrieve any file that could be represented as pandas data frame. The data module contains two parts

- Test part
- Real time part

Test part contains the code to retrieve a pre-defined test data file for testing the GUI. Real time part contains the code to retrieve the real time data file for executing in GUI. Test part code is as follows

```
def test_data(self):
    return pd.read_csv(r'C:\Users\sdsar\OneDrive\Desktop\major
project\data\test\data.csv', encoding='cp1252')
```

Real part code is as follows

```
def real_time(self, file):
    return pd.read_csv(file, encoding='cp1252')
```

The data module also consists the code related to process the raw data and take the only relevant parts of data to send to the GUI. It is as follows

```
def prep(self):
    dt = self.d
    x = dt.iloc[:, [4, 7, 8, 9, 12, 13, 14, 16, 17, 20, 21, 22, 23,
24, 25, 27, 28, 30, 31, 32, 33, 34, 35, 36, 38,
                    40, 41, 43, 44, 45, 46, 48, 49, 51, 52, 53, 54,
57, 71, 72, 73, 74, 75, 76, 77, 78, 80]].values
    y = dt.iloc[:, [1, 2, 3, 4]].values
    return x, y
```

The numbers in ‘dt.iloc’ statement are the attributes in retrieved data that must be passed to GUI

The above codes are coordinated through the constructor of the class in which they operate. The constructor takes parameters like key and file name which are both set to ‘NULL’ by default. Based on the key the data module decides whether to pass test data or real time data to the GUI.

The constructor code is as follows,

```
def __init__(self, k='', file=''):
    self.key = k
    if self.key == 'test':
        self.d = self.test_data()
    else:
        self.d = self.real_time(file)
    self.x, self.y = self.prep()
```

Here if key is ‘test’ then test data is passed else real time data is passed after processing them through the ‘prep ()’ function code.

The complete data module code is as follows

```
import pandas as pd


class data_call:
    def __init__(self, k='', file=''):
        self.key = k
        if self.key == 'test':
            self.d = self.test_data()



        else:
            self.d = self.real_time(file)
        self.x, self.y = self.prep()

    def test_data(self):
        return pd.read_csv(r'C:\Users\sdsar\OneDrive\Desktop\major
project\data\test\data.csv', encoding='cp1252')

    def real_time(self, file):
        return pd.read_csv(file, encoding='cp1252')

    def prep(self):
        dt = self.d
        x = dt.iloc[:, [4, 7, 8, 9, 12, 13, 14, 16, 17, 20, 21, 22,
23, 24, 25, 27, 28, 30, 31, 32, 33, 34, 35, 36, 38,
                       40, 41, 43, 44, 45, 46, 48, 49, 51, 52, 53,
54, 57, 71, 72, 73, 74, 75, 76, 77, 78, 80]].values
        y = dt.iloc[:, [1, 2, 3, 4]].values
        return x, y
```

6.2 Detector Module

Detector module works in coordination with GUI and forms a very important part of GUI. Its main task is to load the model and keep it ready and when the data is passed to it makes use of the model and do predictions.

The detector module is an important part of the GUI. Through this module the GUI executes the task for which it is made.

The detector module contains two parts of code. One code is an initialization constructor code used in python to initialize the class variables.
Its code is as follows,

```
def __init__(self):
    filename = r'C:\Users\sdsar\OneDrive\Desktop\major
project\model\Final_Model.sav'
    self.model = pickle.load(open(filename, 'rb'))
```

The variable 'filename' stores the model location in the local computer which is passed to pickle.load () function and model is retrieved and stored under self.model variable.
The second code consists of the command executed by the model to do predictions.

```
def eval(self, x):
    return self.model.predict(x)
```

The command to make predictions is predict (x) where x contains the set of parameters to be passed to the model.
The complete detector module is as follows,

```
import pickle
class detect:
    def __init__(self):
        filename = r'C:\Users\sdsar\OneDrive\Desktop\major
project\model\Final_Model.sav'

        self.model = pickle.load(open(filename, 'rb'))

    def eval(self, x):
        return self.model.predict(x)
```

6.3 Pre-processor module

It is an important module in case of model training phase. This module helps in processing the training data to be fed to the model for training. In this module, data undergoes various steps of cleaning and feature selection.

The first step in this module is to reduce the data into 32-bit format from 64-bit format so as to facilitate higher computational speeds and low memory usage. The code to perform this is as follows,

```
integer = []
f = []
for i in tqdm(df.columns[:-1]):
    if df[i].dtype == "int64":
        integer.append(i)
    else:
        f.append(i)

df[integer] = df[integer].astype("int32")
df[f] = df[f].astype("float32")
```

The next step is to remove single variable features to reduce outlier formations. The code to perform this is as follows,

```
one_variable_list = []
for i in tqdm(df.columns):
    if df[i].value_counts().nunique() < 2:
        one_variable_list.append(i)
df.drop(one_variable_list, axis=1, inplace=True)
```

The last line of code is generally used to drop columns from a data frame. Then any left-out garbage columns are removed using following code,

```
df.columns = df.columns.str.strip()
```

The next step is to remove infinite or null value rows using following code.

```
df = df[~df.isin([np.nan, np.inf, -np.inf]).any(1)]
```

Then the labels that are similar or come under one category and have less instances carefully are combined using following code

```
df["Label"] = df["Label"].replace(
    ["Web Attack ï¿½ Brute Force", "Web Attack ï¿½ XSS", "Web Attack
ï¿½ Sql Injection"], "Web Attack")
```

Then duplicates are dropped using following code,

```
df = df.drop_duplicates(keep="first")

df.reset_index(drop=True, inplace=True)
```

Now the feature reduction process starts. First highly correlated features are separated out from main data copy then from this separated data upper triangle of correlation matrix is considered and features with correlation greater than 0.95 are found and then those features are dropped from main data. The code for above process is as follows,

```
corr_matrix = df.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
k=1).astype(np.bool))

to_drop = [column for column in upper.columns if any(upper[column] >
0.95)]

df = df.drop(to_drop, axis=1)
```

The complete module code is as follows,

```
import numpy as np
from tqdm import tqdm
def preprocessor(df: object):
    integer = []
    f = []
    for i in tqdm(df.columns[:-1]):
        if df[i].dtype == "int64":
            integer.append(i)
```

```
        else:
            f.append(i)
    df[integer] = df[integer].astype("int32")
    df[f] = df[f].astype("float32")
    one_variable_list = []
    for i in tqdm(df.columns):
        if df[i].value_counts().nunique() < 2:
            one_variable_list.append(i)
    df.drop(one_variable_list, axis=1, inplace=True)

    df.columns = df.columns.str.strip()

    df = df[~df.isin([np.nan, np.inf, -np.inf]).any(1)]

    df["Label"] = df["Label"].replace(
        ["Web Attack ï¿½ Brute Force", "Web Attack ï¿½ XSS", "Web
Attack ï¿½ Sql Injection"], "Web Attack")
    df["Label"] = df["Label"].replace(["DoS GoldenEye", "DoS Hulk",
"DoS Slowhttptest", "DoS slowloris"], "DoS")

    df = df.drop_duplicates(keep="first")

    df.reset_index(drop=True, inplace=True)

    corr_matrix = df.corr().abs()

    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
k=1).astype(np.bool))

    to_drop = [column for column in upper.columns if
any(upper[column] > 0.95)]
    df = df.drop(to_drop, axis=1)

    return df
```

6.4 Model Training

This is the core module from where the complete training code is executed. It contains following two parts,

- Model part
- Train part

Model part consists of different classes each representing a model to be trained. Its code is as follows,

```
from sklearn.ensemble import RandomForestClassifier,
ExtraTreesClassifier, VotingClassifier
from sklearn.metrics import *
from sklearn.svm import SVC
from xgboost import XGBClassifier

class XGBoost:
    def __init__(self):
        self.xgb = XGBClassifier(random_state=42)
    def fitting(self, x, y):
        self.xgb.fit(x, y)



    def predicting(self, x):
        self.preds = self.xgb.predict(x)
    def Metrics(self, y_test):
        return accuracy_score(y_test, self.preds)

class RF:
    def __init__(self):
        self.rf = RandomForestClassifier(random_state=42)

    def fitting(self, x, y):
        self.rf.fit(x, y)

    def predicting(self, x):
        self.preds = self.rf.predict(x)

    def Metrics(self, y_test):
        return accuracy_score(y_test, self.preds)
```

```
class ExtraTrees:
    def __init__(self):
        self.etc = ExtraTreesClassifier(n_estimators=10,
criterion='entropy', random_state=42)

    def fitting(self, x, y):
        self.etc.fit(x, y)

    def predicting(self, x):
        self.preds = self.etc.predict(x)

    def Metrics(self, y_test):
        return accuracy_score(y_test, self.preds)

class VotingClassification:
    def __init__(self, model_instances):
        self.vc = VotingClassifier(estimators=model_instances,
voting='hard', n_jobs=10)

    def fitting(self, x, y):
        self.vc.fit(x, y)

    def predicting(self, x):
        self.preds = self.vc.predict(x)

    def Metrics(self, y_test):
        return accuracy_score(y_test, self.preds),
classification_report(y_test, self.preds)
```

The train part of module consists of models class whose task is to call each class from model part to train on the data. The constructor of models class calls train data module for obtaining processed data which is split into train and test set. The 'train' method of the models class calls each model class of the model part to execute. Then the 'model_instance' method is used to select the best possible model based on the accuracy based on the following code,

```
val = 0
for i in tqdm(self.dic.values()):
```

```
    if val < i[0]:
        clf = i[1]
        val = i[0]
return clf
```

The complete code for this part is,

```
from tqdm import tqdm
from Model_Training.Model import *
from Train_data import Train_set

class models:
    def __init__(self):
        self.dic = {}
        t = Train_set()
        self.x_train, self.x_test, self.y_train, self.y_test = t.data
    def train(self):
        print('XGBoost:')
        xg = XGBoost()
        xg.fitting(self.x_train, self.y_train)
        xg.predicting(self.x_test)
        acc, report = xg.Metrics(self.y_test)
        print('Accuracy: ', acc)
        self.dic['XGBoost'] = [acc, xg.xgb]
        print('\n\nRandom Forest:')
        r = RF()
        r.fitting(self.x_train, self.y_train)
        r.predicting(self.x_test)
        acc, report = r.Metrics(self.y_test)
        print('Accuracy: ', acc)
        self.dic['RandomForest'] = [acc, r.rf]
        print('\n\nExtra Trees Classifier:')
        e = ExtraTrees()
        e.fitting(self.x_train, self.y_train)
        e.predicting(self.x_test)
        acc, report = e.Metrics(self.y_test)
        print('Accuracy: ', acc)
        self.dic['ExtraTrees'] = [acc, e.etc]
        x1 = XGBoost()
        r1 = RF()
```

```
        e1 = ExtraTrees()
        print("\n\nVoting Classifier:")

        model_combinations = [[('ETC', e1.etc), ('RF', r1.rf)],
[('XG', x1.xgb), ('RF', r1.rf)],
                              [('ETC', e1.etc), ('XG', x1.xgb)],
[('ETC', e1.etc), ('RF', r1.rf), ('XG', x1.xgb)]]
        for i in tqdm(range(0, len(model_combinations))):
            print('\n  Voting Classifier', i + 1, ':')
            v = VotingClassification(model_combinations[i])
            v.fitting(self.x_train, self.y_train)
            v.predicting(self.x_test)
            acc, report = v.Metrics(self.y_test)
            print('Accuracy: ', acc)
            self.dic['VC' + str(i + 1)] = [acc, v.vc]
    @property
    def model_instance(self):
        val = 0
        for i in tqdm(self.dic.values()):
            if val < i[0]:
                clf = i[1]
                val = i[0]
        return clf
```

6.5 Train Data Module

This module plays one of the key roles in model training. It provides the data to model. The model training module calls for this module from its train part constructor to obtain the data from local system. This module consists of a constructor for 'Train_set' class that collects all data from the local system and arranges them in proper format. The 'data' method is a property method that when called calls for pre-processor module to process data and then splits the data into train and test set and the independent viable set of train and test sets are transformed using standard scalar property and finally train and test sets are returned.

The complete code is as follows,

```
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tqdm import tqdm
import pandas as pd
from Preprocessor import preprocessor
class Train_set:
    def __init__(self):
        csv_files = []
        s = r'C:\Users\sdsar\OneDrive\Desktop\major
project\data\MachineLearningCSV\MachineLearningCVE'
        for _, _, filenames in os.walk(s):
            for filename in tqdm(filenames):
                k = r'C:\Users\sdsar\OneDrive\Desktop\major
project\data\MachineLearningCSV\MachineLearningCVE\ '
                k = k[:-1]
                csv_files.append(pd.read_csv(k + filename,
encoding='cp1252'))
        self.df = pd.concat(csv_files)

    @property
    def data(self):
        self.df = preprocessor(self.df)
        print(self.df.shape)
        x = self.df.drop(["Label"], axis=1)
        y = self.df["Label"]

        x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=42, stratify=y)
        scalar = StandardScaler()
        x_train = scalar.fit_transform(x_train)
        x_test = scalar.transform(x_test)
        return x_train, x_test, y_train, y_test
```

6.6 Model Controller

The model controller is a special file that is used to start the training process and save the obtained desired model in the local system at the specified location. Its code is as follows,

```
import pickle
from Model_Training import *

if __name__ == '__main__':
    m = models ()
    m.train()
    final_model = m.model_instance
    filename = r'C:\Users\sdsar\OneDrive\Desktop\major
project\model\Final_Model.sav'
    pickle.dump(final_model, open(filename, 'wb'))
```

6.7 GUI

The GUI is a key part of any system. It helps in user interaction with the system. It controls and coordinates the parts system that generate output. The code for GUI is as follows,

```
import os
import tkinter as tk
from tkinter import ttk, filedialog
from tkinter.messagebox import showinfo
from tkinter.ttk import Label
from matplotlib.backends._backend_tk import NavigationToolbar2Tk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib import pyplot as plt
from Data import *
from Detector import *

def alert(k):
    msg = f"Intrusion Of Type '{k}' Detected
    showinfo(title='Intrusion Alert', message=msg)
```

```
def alert_1():
    msg = f"No file selected"
    showinfo(title='Error', message=msg)
    print(msg)


class App:
    def __init__(self, master):
        self.window = master
        self.window.title('Intrusion Detection System')
        self.window.geometry('1100x1000+300+300')
        self.window.state('zoomed')

        self.filename = ''
        self.data = pd.DataFrame()
        self.dic = {}
        # toolbar
        menubar = tk.Menu(self.window)
        menubar.add_command(label='Start  ',
command=self.start,font="Courier 10")
        menubar.add_command(label='Stop  ',
command=self.stop,font="Courier 10")
        menubar.add_command(label='Test  ',
command=self.test,font="Courier 10")
        menubar.add_command(label='Report   ',
command=self.report,font="Courier 10")
        self.window.config(menu=menubar)
        # label
        ttk.Label(self.window, text="Browse File :",
                  font=("Times New Roman", 10)).grid(column=0, row=5,
padx=10, pady=25)

        self.button_explore = tk.Button(master, text="browse",
command=self.view_file)
```

```
        ini = "\n {:<18}   {:<18}   {:<18}   {:<18}
{:<18}\n".format("Source IP", "Source Port", "Destination IP",

"Destination Port", "Label")

        self.z = Label(self.window, text="")
        self.window.grid_propagate(False)
        self.z1 = tk.Text(self.window, height=32, width=100)
        self.z1.insert(tk.END, ini)
        self.z1.grid(row=0, column=0, sticky="nsew")

        # create a Scrollbar and associate it with txt
        scrollb = ttk.Scrollbar(self.window, command=self.z1.yview)
        scrollb.grid(row=0, column=2, sticky='nsew')
        self.z1['yscrollcommand'] = scrollb.set
        self.z.grid(column=3, row=5)
        self.button_explore.grid(column=1, row=5)
        self.z1.grid(column=1, row=15)
        scrollb.grid(column=2, row=15, sticky='nsew')

    def start(self):
        if self.filename != '':
            print(self.filename)
            self.data = data_call(file=self.filename)
            self.process()
        else:
            alert_1()

    def view_file(self):
        self.filename = filedialog.askopenfilename(initialdir="/",
title="Select a File", filetypes=(("CSV Files", "*.csv*"), ("all
files", "*.*")))
        self.z.config(text=str(self.filename), font=("Times New
Roman", 10))

    @staticmethod
    def stop():
        exit(0)
```

```
    def report(self):
        window = tk.Tk()
        # setting the title
        window.title('Report')
        window.geometry("500x500")
        fig, (ax1, ax2) = plt.subplots(2, 1)
        ax1.pie(self.dic.values(), labels=self.dic.keys())
        # show plot
        ax2.axis('off')
        s = sum(self.dic.values()) - self.dic['No Intrusion']
        st = '\n\nNumber of Intruders: ' + str(s) + "\n" + 'Number of
Non-Intruders: ' + str(self.dic['No Intrusion'])
        ax2.text(0.25, 0.95, st, transform=ax2.transAxes,
fontsize=14, verticalalignment='top')
        canvas = FigureCanvasTkAgg(fig, master=window)
        canvas.draw()

        # placing the canvas on the Tkinter window
        canvas.get_tk_widget().pack()

        # creating the Matplotlib toolbar
        toolbar = NavigationToolbar2Tk(canvas, window)
        toolbar.update()
        # placing the toolbar on the Tkinter window
        canvas.get_tk_widget().pack()
        window.mainloop()

    def test(self):
        self.data = data_call('test')
        self.process()

    def process(self):
        detector = detect()
        st = ""
        for i in range(len(self.data.x)):
            k = detector.eval(self.data.x[i].reshape(1, -1))[0]
            if k == 'BENIGN':
                k = 'No Intrusion'
```

```
            if k not in self.dic:
                self.dic[k] = 0
            self.dic[k] += 1
            if k != 'No Intrusion':
                alert(k)
            else:
                st = st + " {:<18}   {:<18}   {:<18}   {:<18}
{:<18}\n".format(self.data.y[i][0], self.data.y[i][1],

self.data.y[i][2], self.data.y[i][3], k)
                self.z1.insert(tk.END, st)
        print(self.dic)

if __name__ == '__main__':
    root = tk.Tk()
    App(root)
    root.mainloop()
```

Chapter 7

IMPLEMENTATION

7.1 Dataset Loading and Pre-processing

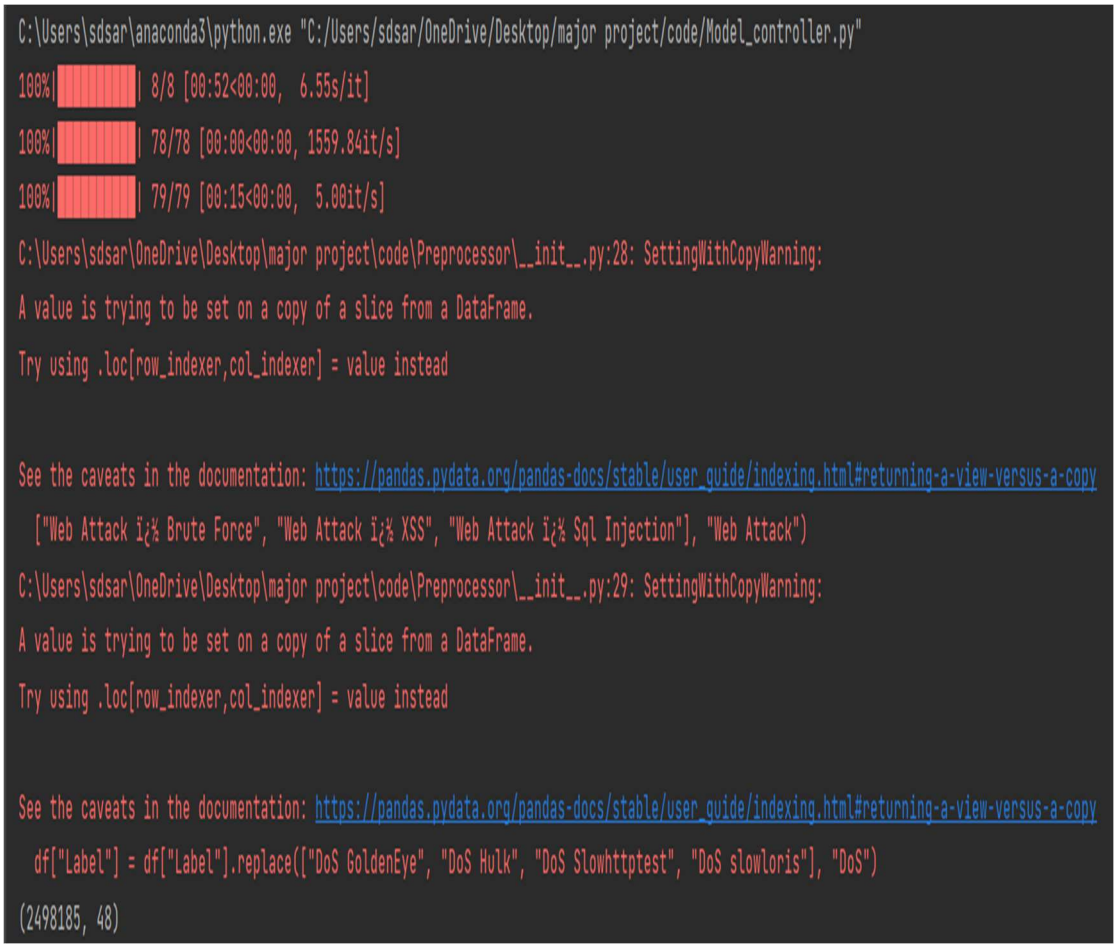


Fig 7.1 Dataset Loading

7.2 Model Training

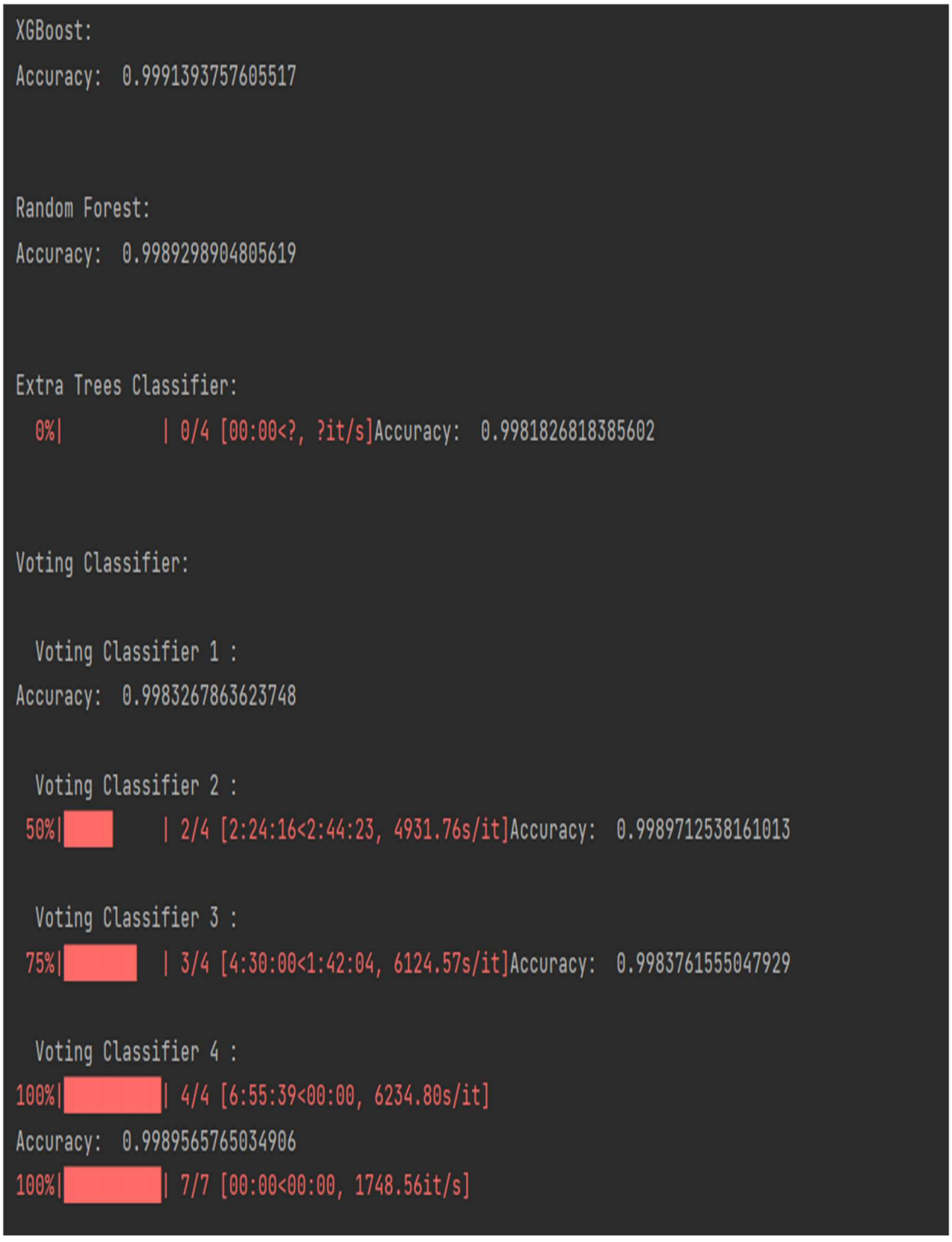


Fig 7.2 Model Training

7.3 Comparison of Different Models Based on Accuracies

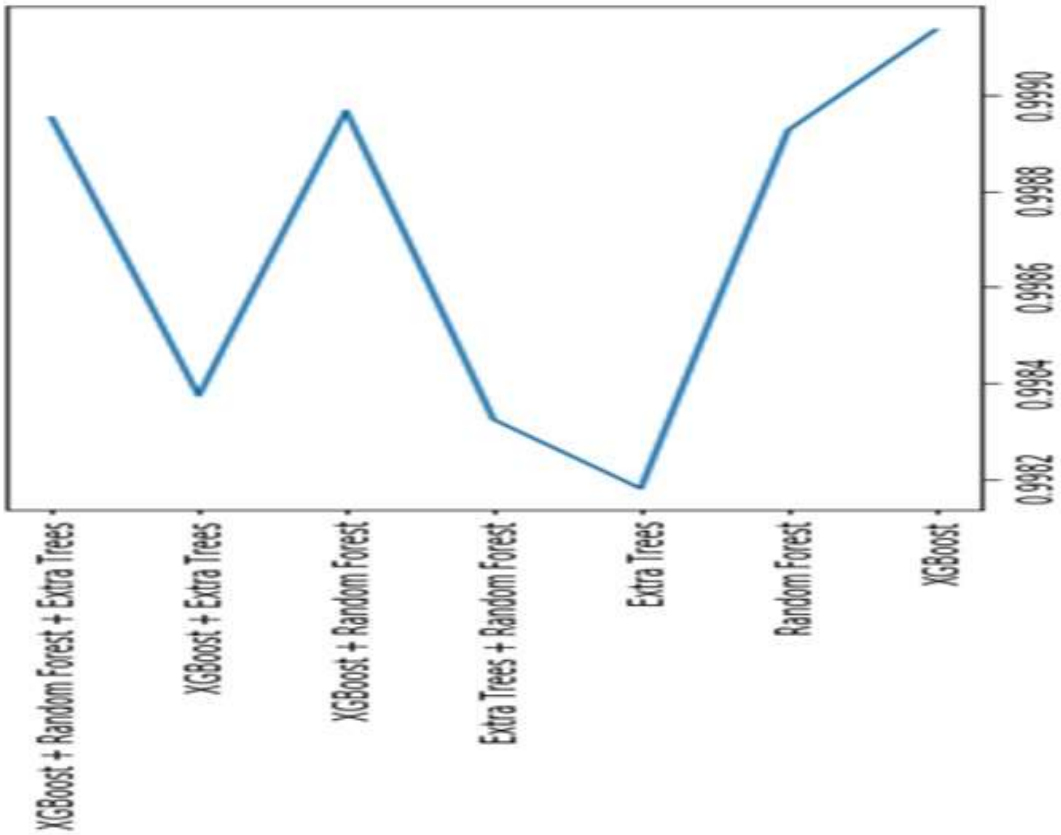


Fig 7.3 Comparison of Models

7.4 GUI Output For Intrusion Detected

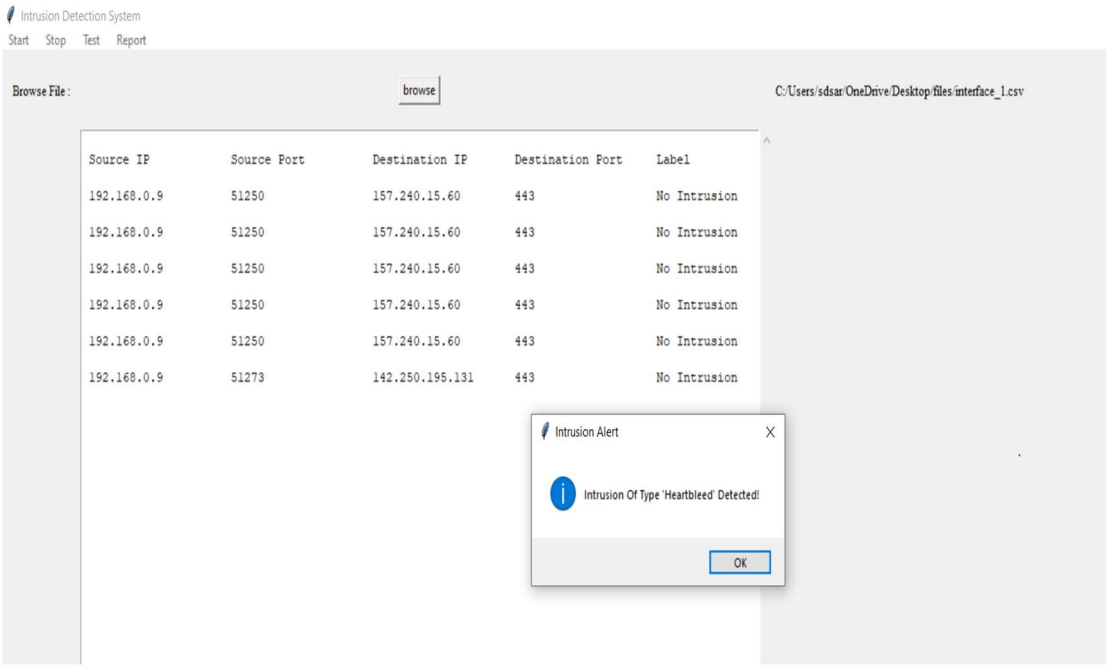


Fig 7.4 GUI Output for Intrusion Detected

7.5 GUI Output For Report

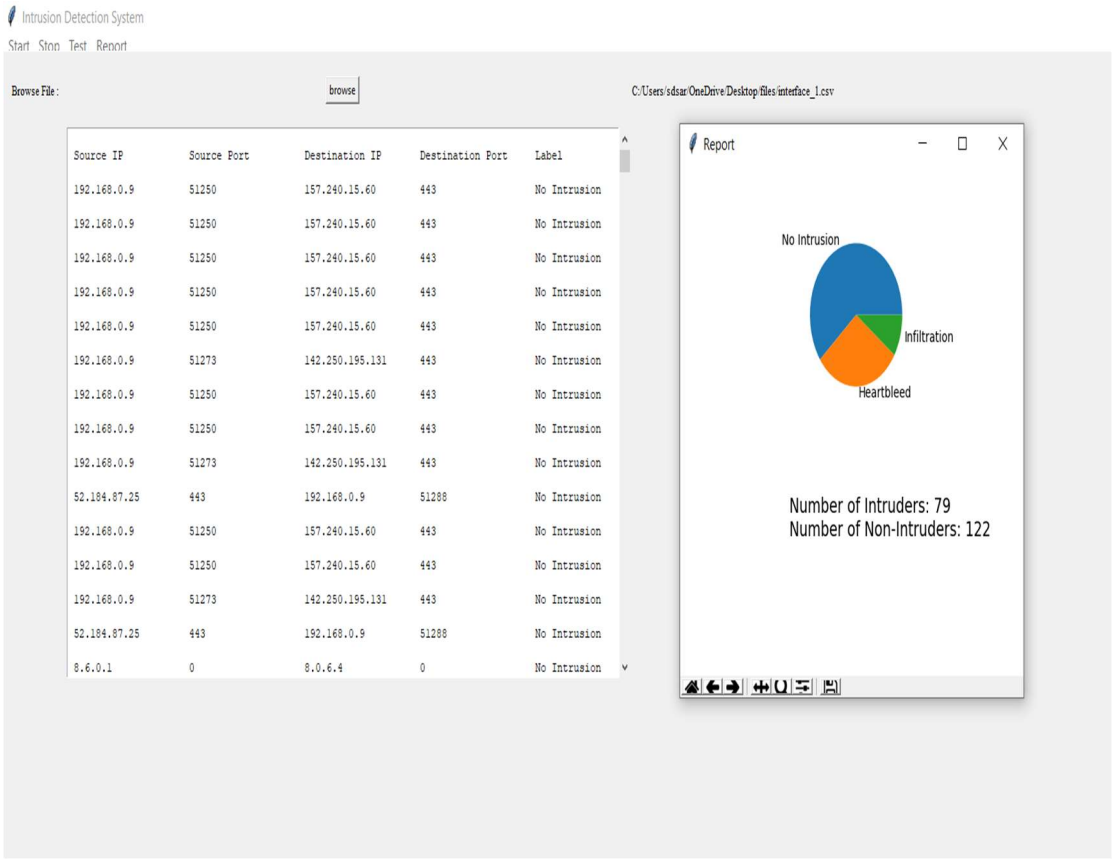


Fig 7.5 GUI Output for Report

Chapter 8

TESTING

8.1 Introduction

Software validation is achieved through a series of tests that demonstrate conformity with requirements. Validation succeeds when software functions in a manner that can be reasonably expected by the customer. Here line by line checking is used to find errors. Comment line facility is used for checking errors.

Testing is necessary for the success of the system. During testing, program to be tested is executed with a set of test data and the output of the program for test data is evaluated to determine if the programs are performing as expected. Validation means checking the quality of software in both simulated and live environments. System validation ensures that the user can in fact match his/her claims, especially system performance.

True validation is verified by having each system tested. First the application goes through a phase often referred as alpha testing in which the errors and failures based on simulated user requirements are verified and studied. The modified software is then subjected to phase two called beta testing in the actual user's site or live environment. After a scheduled time, failures and errors are documented for final correction and enhancements are made before the package is released. In a software development project, errors can be injected at any stage during development.

Even if error detecting and eliminating techniques were employed in the previous analysis and design phases, errors are likely to remain undetected. Unfortunately, these errors will be reflected in the code. Since code is frequently the only product that can be executed and whose actual behaviour can be observed, testing is the phase where the errors remaining from the earlier phases must be detected in addition to detecting the errors introduced during coding activity.

Having proper test cases is central to successful testing. We would like to determine a set of test cases such that successful execution of all of them implies that there are no errors in the program. Therefore, our project crew aimed at selecting the test cases such that the maximum possible numbers of errors are detected by the minimum number of test cases.

8.2 Model Testing On A Set Of Processed Values

x_values	y_values	y_pred
[-0.43626193 -0.47335421 -0.01055812 -0.05128725 -0.3071	BENIGN	BENIGN
[1.59251733e+00 -4.73358000e-01 -1.18527579e-02 -5.128	BENIGN	BENIGN
[-0.45670646 -0.46863802 -0.01055812 -0.04579804 -0.2635	BENIGN	BENIGN
[-0.45670646 -0.47335548 -0.01055812 -0.04380196 -0.2476	BENIGN	BENIGN
[-4.56706462e-01 -4.72495453e-01 -1.18527579e-02 -4.413	BENIGN	BENIGN
[-4.56706462e-01 -4.71000416e-01 -7.96885830e-03 -3.764	BENIGN	BENIGN
[-4.55291071e-01 -4.70220391e-01 -9.26349151e-03 4.044	BENIGN	BENIGN
[-4.56706462e-01 -4.73354209e-01 -1.05581247e-02 -4.380	BENIGN	BENIGN
[2.54208730e+00 -4.72867246e-01 -1.05581247e-02 -2.799	BENIGN	BENIGN
[-0.20534354 -0.47335811 -0.01185276 -0.05128725 -0.3071	PortScan	PortScan
[2.26273109e+00 -4.73012178e-01 -9.26349151e-03 -4.979	BENIGN	BENIGN
[-4.36261928e-01 -4.73352144e-01 -9.26349151e-03 -5.128	BENIGN	BENIGN
[2.37554298e+00 -4.73356897e-01 -1.18527579e-02 -5.078	BENIGN	BENIGN
[-0.45670646 -0.44502877 -0.01185276 -0.04796045 -0.2542	BENIGN	BENIGN
[-0.43626193 1.28837434 0.00497747 0.01441695 0.23584	BENIGN	BENIGN
[-0.45670646 -0.47246507 -0.01185276 -0.04671291 -0.2344	BENIGN	BENIGN
[-0.45670646 -0.46783197 -0.00796886 -0.03897811 -0.2582	BENIGN	BENIGN
[-0.45670646 -0.47242702 -0.01185276 -0.04704558 -0.2397	BENIGN	BENIGN
[-0.45670646 -0.47248699 -0.01055812 -0.04546536 -0.2608	BENIGN	BENIGN
[-0.45670646 -0.47334714 -0.01185276 -0.04762777 -0.2489	BENIGN	BENIGN
[-4.55291071e-01 2.36387970e+00 -6.67422509e-03 -2.384	DoS	DoS

Fig 8.1 Model Testing

8.3 GUI Testing

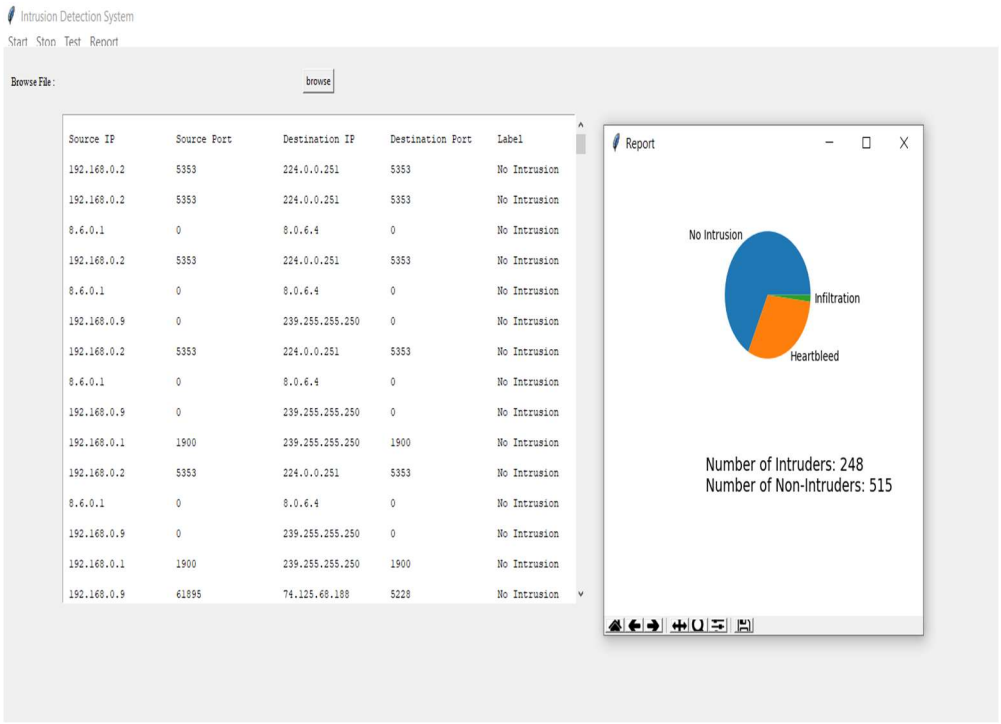


Fig 8.2 GUI Testing

Chapter 9

CONCLUSION

Intrusion detection system (IDS) is one of the extensively used techniques in a network topology to safeguard the integrity and availability of sensitive assets in the protected systems. A Network Intrusion Detection System (NIDS) helps system administrators to detect network security breaches in their organizations

The network intrusion detection is implemented mostly using anomaly-based techniques which are machine learning based approaches that identify patterns in network anomaly dataset which contains different scenarios of attacks. Most of the machine learning algorithms that are used to detect intrusion are supervised algorithms that work on the datasets like NSL-KDD dataset. Some of the commonly used algorithms are Random Forest classifier, Naïve Bayes classifier, Decision Table classifier and Support vector machines. Though there is no single machine learning algorithm which can handle efficiently all the types of attacks, the random forest classifier presents acceptable performance parameters except the false negative parameter, the support vector machines could register a notable accuracy range of 93 to 98 percentage. Though most of the above techniques were single model algorithms, few of them are extremely slow and unstable and prone to generate more false positives and false negatives.

Hence, to improve the stability of the model and to achieve higher accuracy rate, we propose to implement multi-model approaches like ensemble classifiers. Ensemble models are highly stable models, have high computational rates and are highly flexible in nature. Ensemble methods helps improve machine learning results by combining multiple models. Using ensemble methods allows to produce better predictions compared to a single model. The proposed system carried out an ensemble of Random forest classifier, Xgboost algorithm and Extra tress classifier of which, xgboost alone could predict intrusions with highest accuracy of 99.91 percentage. Besides that, the best accuracies of Random Forest classifier, Extra trees classifier and combinations of xgboost, extra trees and random forest classifier were 99.89%, 99.81% and 99.89% respectively.

Use of ensemble classifier enabled model to be trained on multiple combinations of attributes which in turn helped in detecting the patterns or anomalies in the network data more effectively which resulted in a highly accurate and highly stable model.

Chapter 10

BIBLIOGRAPHY

[1] Jabez J, Dr.B.Muthukumar, "Intrusion Detection System (IDS): Anomaly Detection using Outlier Detection Approach"

[2] Hervé Debar, "An Introduction to Intrusion-Detection Systems"

[3] TONGTONG SU, HUAZHI SUN, JINQI ZHU, SHENG WANG and YABO LI, "Deep Learning Methods on Network Intrusion Detection using NSL-KDD dataset"

[4] Mostofa Ahsan and Kendall E. Nygard "Convolutional Neural Networks with LSTM for Intrusion Detection"

[5] Yalei Ding, Yuqing Zhai, "Intrusion Detection System for NSL-KDD Dataset Using Convolutional Neural Networks"

[6] Quamar Niyaz, Weiqing Sun, Ahmad Y Javaid, and Mansoor Alam, "A Deep Learning Approach for Network Intrusion Detection System"

[7] Anazida Zainal, Mohd Aizaini Maarof, Siti Mariyam Shamsuddin, "Ensemble classifiers for network intrusion detection system"

[8] Prof. Waweru Mwangi, Dr. Otieno Calvin, "Ensemble Network Intrusion Detection Model Based on Classification & Clustering for Dynamic Environment"

[9] Raihan Masud, Hossen A. Mustafa, "Network Intrusion Detection System Using Voting Ensemble Machine Learning"

[10] Mohit Tiwari, Raj Kumar, Akash Bharti, Jai Kishan, "INTRUSION DETECTION SYSTEM"

[11] Anish Halimaa A, Dr. K.Sundarakantham, "MACHINE LEARNING BASED INTRUSION DETECTION SYSTEM"

[12] Gopalkrishna N. Prabhu, Kushank Jain, Nandan Lawande, Narendra Kumar,Yashasvi Zutshi, Rohan Singh, Jyoti Chinchole "NETWORK INTRUSION DETECTION SYSTEM"

[13] Iman Sharafaldin, Arash Habibi Lashkari and Ali A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization"

[14] Monowar H. Bhuyan, Dhruba K. Bhattacharyya, and Jugal K. Kalita "Towards Generating Real-life Datasets for Network Intrusion Detection"

Chapter 11

APPENDIX

PyCharm:

PyCharm is an integrated development environment (IDE) used in computer programming, specifically for the Python language. It is developed by the Czech company JetBrains. It provides code analysis, a graphical debugger, an integrated unit tester, integration with version control systems (VCSes), and supports web development with Django as well as data science with Anaconda.

Its features are:

- Coding assistance and analysis, with code completion, syntax and error highlighting, linter integration, and quick fixes
- Project and code navigation: specialized project views, file structure views and quick jumping between files, classes, methods, and usages
- Python refactoring: includes rename, extract method, introduce variable, introduce constant, pull up, push down and others.
- Support for web frameworks: Django, web2py and Flask
- Integrated Python debugger
- Integrated unit testing, with line-by-line code coverage
- Google App Engine Python development [professional edition only]
- Support for scientific tools like Matplotlib, NumPy and SciPy

Standard Scalar:

Standard scaler standardizes a feature by subtracting the mean and then scaling to unit variance. Unit variance means dividing all the values by the standard deviation. Standard scaler does not meet the strict definition of *scale* I introduced earlier.

Standard scaler results in a distribution with a standard deviation equal to 1. The variance is equal to 1 also, because *variance = standard deviation squared.*

Standard scaler makes the mean of the distribution 0. About 68% of the values will lie be between -1 and 1

Correlation Matrix:

A correlation matrix is a table showing correlation coefficients between variables. Each cell in the table shows the correlation between two variables. A correlation matrix is used to summarize data, as an input into a more advanced analysis, and as a diagnostic for advanced analyses.

Key decisions to be made when creating a correlation matrix include:

- choice of correlation statistic
- coding of the variables
- treatment of missing data
- presentation

CICFLOW Meter:

CICFlow Meter is a network traffic flow generator and analyser. It can be used to generate bidirectional flows, where the first packet determines the forward (source to destination) and backward (destination to source) directions, hence more than 80 statistical network traffic features such as Duration, Number of packets, Number of bytes, Length of packets, etc. can be calculated separately in the forward and backward directions. Additional functionalities include, selecting features from the list of existing features, adding new features, and controlling the duration of flow timeout. The output of the application is the CSV format file that has six columns labeled for each flow (FlowID, SourceIP, DestinationIP, SourcePort, DestinationPort, and Protocol) with more than 80 network traffic analysis features.

The steps for installation are:

- Clone the project using https://github.com/ahlashkari/CICFlowMeter.git.
- Import the project to a Java IDE.
- Install MVN package using following code.

```
# Open the Terminal in IDEA
$ cd pathtoproject/jnetpcap/linux/jnetpcap-1.4.r1425
 # Installation
$ mvn install:install-file -Dfile=jnetpcap.jar -DgroupId=org.jnetpcap -DartifactId=jnetpcap -Dversion=1.4.1 -Dpackaging=jar
```

- Open project file build.gradle and modify repositories

```
repositories {
    mavenLocal()
        // maven central source
    //  mavenCentral()
        // maven Aliyuan
        maven { url 'http://maven.aliyun.com/nexus/content/groups/public' }
}
```

- Build the project and run.